



**UNIVERSIDADE FEDERAL
DE ALAGOAS**

**UNIVERSIDADE FEDERAL DE ALAGOAS
INSTITUTO DE COMPUTAÇÃO
CIÊNCIA DA COMPUTAÇÃO**

Framework para Compiladores e Interpretadores Usando Microsserviços

Relatório Técnico Final Integrado

Alan Diogo da Rocha Oliveira - *Aluno de Compiladores*

Bruno Gomes de Barros Filho - *Aluno de Compiladores, Reuso de software e metodologias ágeis e Tóp em eng de software: projetando linhas de prod de software*

Karllison Henrique da Silva - *Aluno de Tóp em eng de Software: projetando linhas de prod de software*

Maria Aparecida da Silva Nascimento - *Aluna de Compiladores*

Professor Orientador:

Arturo Hernandez Dominguez

Código-fonte (GitHub):

<https://github.com/brunog2/minipar-framework>

📄 Apresentação Minipar Framework 2026.1

MACEIÓ, AL
2026

Sumário

1. Introdução.....	5
1.1 Enunciado do projeto.....	5
1.2 Objetivos.....	5
2. Metodologia Ágil e Planejamento.....	6
2.1 Scrum e Sprints.....	6
2.2 Backlogs.....	6
3. Arquitetura do Sistema.....	6
3.1 Arquitetura de Microserviços.....	6
3.2 Gestão da Variabilidade (LPS).....	8
3.3 Conceitos: Frozen-spot, Hotspot e Método Abstrato.....	9
3.4 Princípio Hollywood e Inversão de Controle.....	10
3.5 Esqueleto AbstractBackendTranslator.....	10
3.6 Framework versus Código de Instância.....	10
3.7 Processo de Criação de Nova Aplicação (6 Passos).....	11
3.8 Instâncias de Referência e Extensão PythonBackend.....	11
3.9 Frontend como Cliente LPS.....	12
3.10 Backend Registry e Princípio Aberto-Fechado (OCP).....	12
3.11 Especificação Detalhada das Interfaces REST.....	12
3.11.1 Gateway Central (api-gateway).....	12
3.11.2 ms-front-end (porta 3001) — POST /parse.....	13
3.11.3 ms-semantic (porta 3002) — POST /analyze.....	13
3.11.4 ms-interpreter (porta 3003) — POST /execute.....	14
3.11.5 ms-codegen-* — POST /generate (portas 3004, 3005, 3007, 3008).....	14
3.11.6 ms-parallel-coord (porta 3006) — POST /coordinate.....	14
3.11.7 Tratamento de Erros e Encadeamento do Pipeline.....	15
4. Modelagem UML.....	15
4.1 Diagrama de Casos de Uso.....	15
4.2 Arquitetura de Componentes.....	16
4.3 Diagrama de Classes.....	16
5. Fases do Compilador MiniPar 2026.1.....	18
5.1 Evolução 2025.1 - 2026.1.....	18
5.2 Gramática BNF Completa com Suporte OO.....	19
5.2.1 Palavras-chave e Tabela de Tokens.....	21
5.2.2 Exemplos de Código Válido.....	21
5.2.3 Restrições Semânticas.....	22
5.3 Pseudocódigos e Implementação dos Analisadores.....	22
5.3.1 Analisador Léxico.....	23

5.3.2	Analizador Sintático e AST.....	23
5.3.3	Coordenador de Paralelismo Distribuído (ms-parallel-coord).....	24
5.3.4	Worker Socket (processo independente).....	25
5.3.5	Analizador Semântico e Tabela de Símbolos.....	25
5.3.6	Emissão de Código C e Invocação do Compilador Externo.....	25
5.4	Representação Intermediária TAC.....	26
5.5	Back-ends de Tradução.....	27
5.5.1	Interpretador.....	27
5.5.2	Back-end C (CBackend).....	27
5.5.3	Back-end C++ (CppBackend).....	28
5.5.4	Back-end Rust (RustBackend).....	28
5.5.5	Back-end ARM (ARMBBackend).....	28
5.5.6	Paralelismo par/seq e comunicação por soquetes.....	28
6.	Técnicas de Reuso.....	29
6.1	Engenharia de Domínio Aplicada a Compiladores.....	29
6.2	Padrão Template Method: Análise Profunda.....	29
6.3	Mapa de Reuso 2025.1 - 2026.1.....	30
6.4	Componentes, Acoplamento e Contratos.....	30
6.5	Registry (OCP) e Strategy Implícita.....	31
6.6	Padrões Adicionais Identificados.....	31
6.7	Métricas Narrativas de Reuso.....	31
6.8	Hooks Opcionais (hook_validate, hook_prepare).....	31
6.9	Hotspots Futuros (Roadmap).....	32
6.10	Contrato _template_backend.py.....	32
7.	Implementação.....	32
7.1	Estrutura do Monorepo.....	32
7.2	Tecnologias Utilizadas.....	33
7.3	API Gateway e Microserviços.....	33
7.4	Feature Model e Pontos de Variação.....	35
7.5	Binding Time.....	35
7.6	Matriz de Configurações de Produto.....	35
7.7	Microserviços como Assets Reutilizáveis.....	36
7.8	Docker Compose como Configuração de Produto.....	36
7.9	Decisão LPS: Microserviços versus FeatureIDE.....	36
7.10	Decisão Arquitetural: NestJS versus Spring Boot.....	36
8.	Testes e Resultados.....	36
8.1	Resultados da Fase 1.....	36
8.2	Casos de validação manual.....	37
8.3	Casos de validação — Fase 2 (back-ends).....	37

8.4 Resultados da Fase 2 — Back-ends reais.....	38
8.5 Evidências Visuais da Interface Web.....	38
8.6 Teste de Real Paralelismo (3 Computadores).....	39
8.7 Simulação de Fractal.....	42
8.8 Validação Automatizada.....	43
8.9 Auditoria de Conformidade.....	43
8.10 Casos de Validação — Fase 3.....	44
9. Conclusão.....	44
9.1 Resposta à Crítica "Gerador versus Framework"	45
9.2 Limitações e Trabalhos Futuros.....	45
9.3 Link para o Repositório (GitHub).....	45
9.4 Link para o UI de demonstração e Apresentação.....	45
10. Referências.....	46

1. Introdução

A construção de compiladores e interpretadores é uma das atividades mais fundamentais da ciência da computação, reunindo técnicas de análise formal de linguagens, teoria de autômatos, semântica de programas e geração de código. O projeto MiniPar Framework 2026.1 surge nesse contexto, propondo a integração de três grandes temas: a implementação completa do pipeline de compilação da linguagem MiniPar com suporte à orientação a objetos, a organização do sistema como uma Linha de Produto de Software (LPS) com variabilidade explícita, e a adoção de uma arquitetura de microsserviços que permita o reuso e a substituição independente de cada fase do compilador.

1.1 Enunciado do projeto

Este relatório consolida o trabalho desenvolvido como requisito parcial integrado das disciplinas ministradas pelo Prof. Arturo Hernandez Dominguez no Instituto de Computação da UFAL, abrangendo **Compiladores, Reuso de software e metodologias ágeis e Tópicos em Engenharia de Software: projetando linhas de prod de software**. O artefato central é o MiniPar Framework 2026.1: uma linha de produto de software organizada em microsserviços, destinada ao processamento da linguagem MiniPar com orientação a objetos, construções de paralelismo par/seq e múltiplos back-ends (interpretador, C/C++, Rust, ARM e extensão Python). O repositório mantém documentação de gestão alinhada ao código: COMPLIANCE_AUDIT.md, SCHEDULE.md, ACTIVITIES.md e ROADMAP.md.

1.2 Objetivos

O objetivo principal é o desenvolvimento do MiniPar Framework 2026.1, um sistema distribuído de compilação e interpretação baseado em microsserviços, que evolui a linguagem MiniPar para suporte à orientação a objetos e paralelismo real, validando a arquitetura por meio de testes de execução distribuída em três máquinas. Objetiva-se: (i) completar o suporte OO na gramática e AST; (ii) expor análise léxica, sintática e semântica como serviços HTTP; (iii) materializar os back-ends da LPS como microsserviços que reutilizam minipar-core; e (iv) registrar pontos de variação e variantes que caracterizam a linha de produto.

2. Metodologia Ágil e Planejamento

O desenvolvimento foi conduzido com base nos princípios do framework Scrum, adaptado ao contexto acadêmico. A organização priorizou entregas incrementais, comunicação contínua e rastreabilidade das tarefas por meio do repositório.

2.1 Scrum e Sprints

O projeto foi dividido em sprints curtas, alinhadas aos marcos definidos no cronograma da disciplina. Cada sprint tinha um objetivo e entregável verificável. As reuniões de alinhamento ocorreram de forma assíncrona via repositório, com o arquivo SCHEDULE.md servindo como referência central de planejamento e o board de issues para rastreamento das tarefas em andamento e o backlog atualizado a cada sprint.

2.2 Backlogs

- Product Backlog: gestão da variabilidade LPS (modo interpretador versus compilador, back-end alvo, ambiente local ou distribuído); casos de teste de paralelismo; demonstração do fractal.
- Sprint Backlog Fase 1: pacote minipar-core; microserviços ms-front-end e ms-semantic; Docker Compose com PIPELINE_MODE=http.
- Sprint Backlog Fase 2: módulo translation/ (Template Method); ms-interpreter; ms-codegen-c, ms-codegen-rust, ms-codegen-arm; variável PIPELINE_BACKEND_MODE=http.
- Sprint Backlog Fase 3: ms-parallel-coord; workers socket; fractal Sierpinski (13_sierpinski.minipar); integração DISTRIBUTED_SOCKETS E2E.

3. Arquitetura do Sistema

3.1 Arquitetura de Microserviços

O MiniPar Framework 2026.1 adota uma arquitetura de microserviços na qual cada fase do compilador é implementada como um serviço independente, com responsabilidade bem delimitada, interface REST e comunicação via JSON. Essa escolha não é apenas técnica: ela materializa diretamente os princípios de desacoplamento e variabilidade exigidos pela disciplina de Linhas de Produto de Software, permitindo que cada variante de saída do compilador seja ativada, substituída ou omitida sem impacto sobre os demais componentes.

O ponto central da arquitetura é o API Gateway, implementado em Node.js, que recebe todas as requisições do frontend por meio de endpoint POST /api/v1/process e as roteia

sequencialmente pelos serviços da pipeline. O gateway também é responsável pela persistência do histórico de compilações no PostgreSQL e pela resolução da variabilidade: com base no campo `targetVariability` enviado pelo cliente, ele determina qual back-end será acionado ao final do pipeline.

Os serviços são organizados em duas camadas. A primeira camada, compartilhada por todas as variantes, compreende o `ms-front-end` (porta 3001), responsável pela análise léxica e sintática com geração da AST em JSON, e o `ms-semantic` (porta 3002), responsável pela verificação de tipos, controle de escopo e construção da tabela de símbolos. A segunda camada é variável e corresponde ao back-end selecionado: `ms-interpret` (porta 3003) para execução direta, `ms-codegen-c` (porta 3004) para geração de código C com compilação GCC, `ms-codegen-rust` (porta 3005) para geração Rust, `ms-parallel-coord` (porta 3006) para execução distribuída em três máquinas, e `ms-codegen-arm` (porta 3007) para geração de Assembly ARMv7.

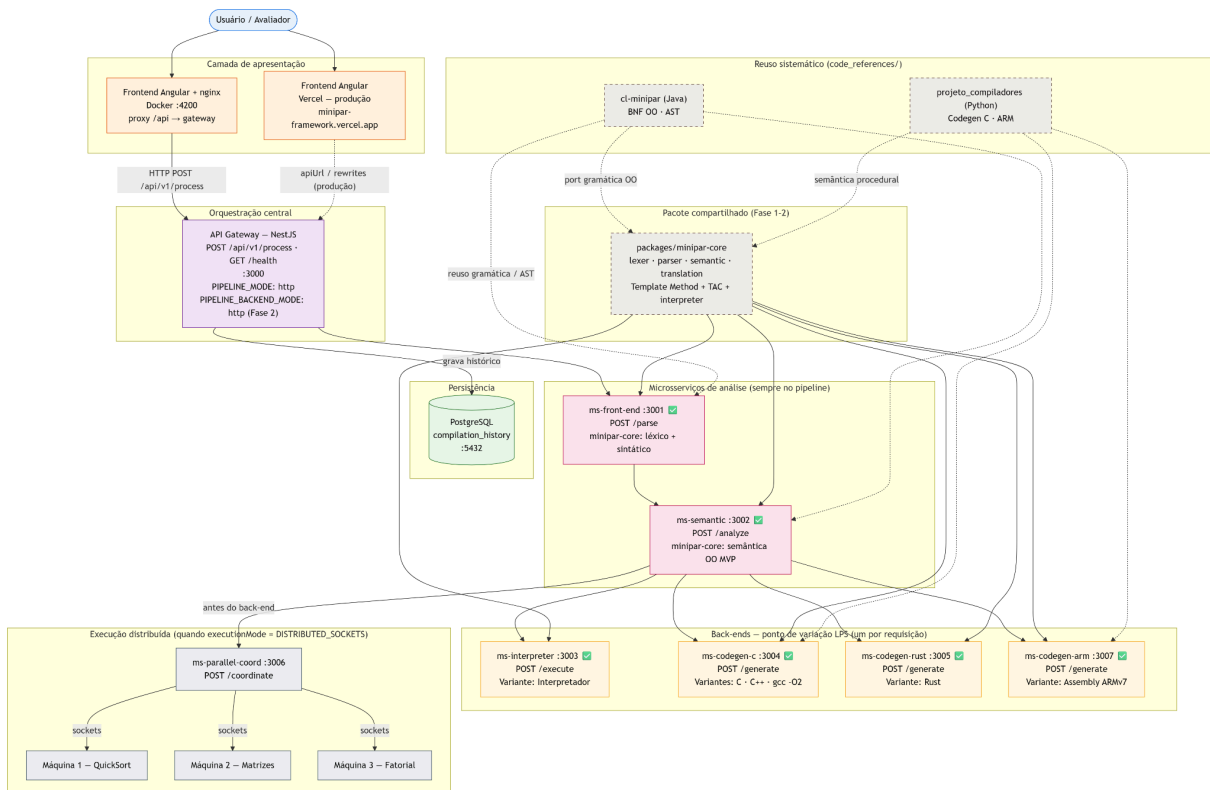


Figura 1: Arquitetura geral do MiniPar Framework, organizando as camadas de apresentação (Angular/Vercel), orquestração (API Gateway), análise estática (ms-front-end e ms-semantic), back-ends LPS, execução distribuída e o pacote compartilhado minipar-core com as referências de reuso.

3.2 Gestão da Variabilidade (LPS)

O framework é documentado e estruturado como uma Linha de Produto de Software com pontos de variação explícitos. O primeiro e principal ponto de variação é o modo de execução, que admite duas variantes mutuamente exclusivas: compilador, que gera código em uma linguagem-alvo e o submete a um compilador externo (GCC ou rustc), e interpretador, que executa a AST diretamente sem geração de código intermediário. O segundo ponto de variação, subordinado ao modo Compilador, é o back-end de compilação, com as variantes C, C++, Rust e Assembly ARMv7. O terceiro ponto de variação é o ambiente de execução, podendo ser Local (execução em uma única máquina) ou Distribuído, no qual o coordenador despacha tarefas para três máquinas via sockets TCP. Um ponto de variação opcional adicional, aplicável apenas aos back-ends C e C++, é o nível de otimização do GCC, parametrizável entre -O0, -O1 e -O2.

O mecanismo de binding dessas variações ocorre em tempo de requisição, sem necessidade de recompilação do framework: o campo `targetVariability` da API determina qual serviço será invocado pelo gateway, tornando a seleção de produto completamente dinâmica.

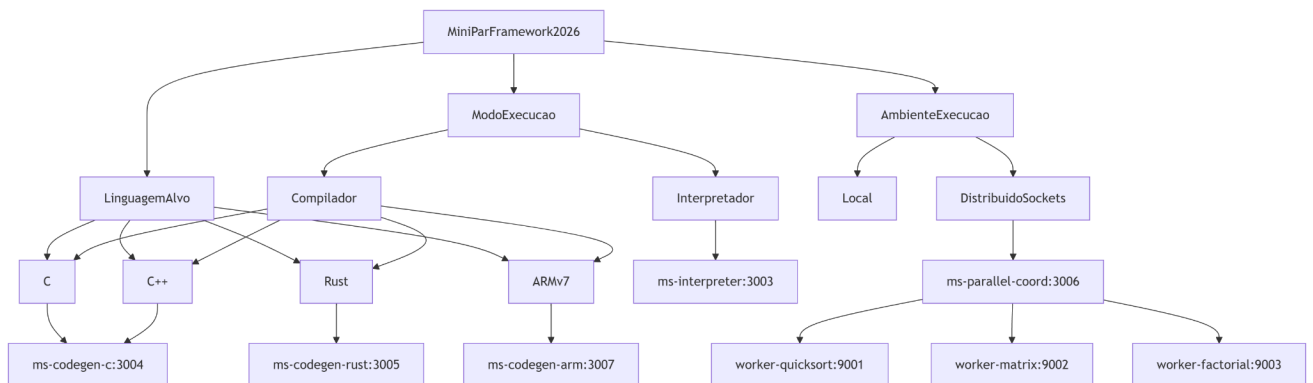


Figura 2: Árvore de features da LPS, com os três pontos de variação do framework: modo de execução (Interpretador ou Compilador), linguagem-alvo (C, C++, Rust, ARMv7) e ambiente de execução (Local ou Distribuído via Sockets), com mapeamento direto para os microserviços responsáveis por cada variante.

3.3 Conceitos: Frozen-spot, Hotspot e Método Abstrato

A terminologia de engenharia de domínio distingue três camadas complementares:

- Frozen-spot: código invariante do framework que o desenvolvedor nunca sobrescreve. Exemplos: `AbstractBackendTranslator.translate()`, `validate()`, `prepare()`, pipeline do gateway, lexer/parser default.

- Método abstrato: contrato imposto pela classe base — assinatura fixa, sem implementação. Exemplos: `emit()`, `finalize()`.
- Hotspot: corpo que o desenvolvedor escreve dentro do método abstrato. Exemplo: `CBackend.emit()` escolhe `TAC→C→gcc -O2`; `Interpreter.emit()` executa a AST diretamente.

Tabela 1: Mapa frozen-spot versus hotspot por camada.

Camada	Frozen-spot (framework)	Hotspot (dev da instância)
minipar-core lexer/parser	<code>Lexer.tokenize()</code> , <code>Parser.parse()</code>	Futuro: lexer com autômatos
Semântica	<code>semantic_json.analyze_ast_dict</code>	—
Tradução	<code>translate()</code> , <code>validate()</code> , <code>prepare()</code>	<code>emit()</code> , <code>finalize()</code>
Gateway	<code>PipelineService</code> pipeline fixo	—
Registry	Estrutura <code>BACKEND_REGISTRY</code>	Nova entrada por variante
Microserviço MS	Contrato <code>REST /generate</code>	Implementação thin wrapper
Frontend Angular	Editor, painel, console	Entrada LPS no feature-panel

3.4 Princípio Hollywood e Inversão de Controle

O Hollywood Principle ("don't call us, we'll call you") materializa-se no fluxo: o api-gateway orquestra o pipeline; o microserviço invoca `Backend.translate()`; o esqueleto chama `emit()` no momento correto. O desenvolvedor do backend não invoca o gateway — o framework invoca o código do dev.

```
const descriptor = BACKEND_REGISTRY.find(
  (b) => b.variability === targetVariability,
);
const backendUrl = this.config.getOrThrow<string>(descriptor.envKey);
const res = await firstValueFrom(
  this.http.post(`${backendUrl}${descriptor.endpoint}`, { ...payload }),
);
```

3.5 Esqueleto AbstractBackendTranslator

O frozen-spot central está em `packages/minipar-core/minipar_core/translation/base_translator.py`:

```
def translate(self, ast_dict: dict) -> TranslationResult:
    self._errors: list[str] = []
    self.validate(ast_dict)
```

```

if self._errors:
    return TranslationResult(output="; ".join(self._errors), exit_code=1)
self.prepare(ast_dict)
self.emit(ast_dict) # HOTSPOT — implementado pelo dev da instância
return self.finalize() # HOTSPOT — monta o resultado final

```

Hooks opcionais `hook_validate()` e `hook_prepare()` permitem validações e pré-processamento sem violar frozen-spots. Se não sobrescritos, comportam-se como no-op. O guia completo está em `packages/minipar-core/EXTENDING.md`.

3.6 Framework versus Código de Instância

O framework compreende os frozen-spots (minipar-core, gateway, contratos REST). O código de instância são os hotspots preenchidos pela equipe: cada back-end concreto (`c_backend.py`, `interpreter.py`, `python_backend.py`) documenta o algoritmo escolhido. As cinco variantes existentes (INTERPRETER, C, CPP, RUST, ASSEMBLY) são instâncias de referência funcionando como demonstração do contrato.

3.7 Processo de Criação de Nova Aplicação (6 Passos)

Para instanciar uma nova aplicação (ex.: compilador MiniPar → Go), o desenvolvedor segue seis passos documentados em `CREATING_AN_APPLICATION.md`:

- Passo 1 — Reusar o chassi: minipar-core, api-gateway, ms-front-end, ms-semantic, Docker Compose base permanecem sem modificação.
- Passo 2 — Criar classe de backend: `GoBackend(AbstractBackendTranslator)` em `minipar_core/translation/`.
- Passo 3 — Implementar hotspots: `emit()` transforma AST/TAC no artefato; `finalize()` compila/executa e monta `TranslationResult`.
- Passo 4 — Criar microsserviço: FastAPI com POST `/generate` espelhando `ms-codegen-c` (~30 linhas de thin wrapper)
- Passo 5 — Registrar variante: uma linha em `backend-registry.ts` + variável `MS_CODEGEN_*` no `.env` + serviço no Compose.
- Passo 6 — Expor na UI: enum `TargetVariability`, type TS e opção no feature-panel (configuração LPS, não algoritmo).

Tabela 2: Três instâncias acadêmicas sobre o mesmo framework.

Instância	Microserviço	Hotspot	Produto
App A — Interpretador	ms-interpreter	Interpreter.emit()	Execução direta AST
App B — Compilador C	ms-codegen-c	CBackend.emit()	C + gcc -O2
App C — Python (extensão)	ms-codegen-python	PythonBackend.emit()	Python + python3

3.8 Instâncias de Referência e Extensão PythonBackend

A extensão PythonBackend demonstra que um desenvolvedor adiciona back-end sem alterar frozen-spots. Implementação em `python_backend.py`; catálogo em `applications/extension-python/`:

```
def emit(self, ast_dict: dict) -> None:
    tac = TACGenerator().lower(ast_dict)
    self._code = SimplePythonCodeGenerator().generate(tac)

def finalize(self) -> TranslationResult:
    result = self._run_python(self._code) # python3 no container
    return TranslationResult(output=result["output"], code=self._code)
```

Registro: { variability: "PYTHON", envKey: "MS_CODEGEN_PYTHON_URL", endpoint: "/generate" }. Fixture: 16_codegen_python.minipar. Evidência: docs/evidence/16_codegen_python_output.txt.

3.9 Frontend como Cliente LPS

O frontend Angular (frontend/) é casca LPS de produto (frozen-spot de infraestrutura), não hotspot de compilação. Componentes code-editor, feature-panel e output-panel disparam POST `/api/v1/process` com `targetVariability` — seleção de variante LPS em binding time de execução, não implementação de `emit()`. Qualquer cliente HTTP (curl, extensão IDE, script Python) pode substituir o Angular, respeitando o contrato mínimo:

```
POST /api/v1/process
{ "sourceCode": "...", "targetVariability": "C", "executionMode": "LOCAL" }
```

3.10 Backend Registry e Princípio Aberto-Fechado (OCP)

O `BACKEND_REGISTRY` central evita switch/case no pipeline. Adicionar backend = nova entrada no array:

```
export const BACKEND_REGISTRY: BackendDescriptor[] = [
  { variability: 'INTERPRETER', envKey: 'MS_INTERPRETER_URL', endpoint: '/execute' },
  { variability: 'C', envKey: 'MS_CODEGEN_C_URL', endpoint: '/generate' },
  { variability: 'CPP', envKey: 'MS_CODEGEN_C_URL', endpoint: '/generate' },
  { variability: 'RUST', envKey: 'MS_CODEGEN_RUST_URL', endpoint: '/generate' },
  { variability: 'ASSEMBLY', envKey: 'MS_CODEGEN_ARM_URL', endpoint: '/generate' },
  { variability: 'PYTHON', envKey: 'MS_CODEGEN_PYTHON_URL', endpoint: '/generate' },
];
```

Nenhum outro arquivo do gateway precisa ser alterado para rotear uma nova variante, apenas registry, env e microserviço correspondente. O endpoint GET /api/v1/variants expõe variantes dinamicamente para evolução OCP completa na UI.

3.11 Especificação Detalhada das Interfaces REST

3.11.1 Gateway Central (api-gateway)

O gateway constitui o ponto único de entrada do framework na porta 3000. O endpoint principal aceita o código-fonte, a variante de tradução desejada e o modo de execução, devolvendo a AST serializada, a tabela de símbolos, o texto de saída, o código gerado quando aplicável, o registro dos passos do pipeline e, no modo distribuído, os resultados agregados dos workers. Os valores admitidos para targetVariability compreendem INTERPRETER, C, CPP, RUST, ASSEMBLY e PYTHON. O campo executionMode aceita LOCAL ou DISTRIBUTED_SOCKETS.

```
// Requisição
{
  "sourceCode": "class Main { void run() { println(\"ok\"); } }",
  "targetVariability": "INTERPRETER",
  "executionMode": "LOCAL"
}
// Resposta
{
  "success": true,
  "targetVariability": "C",
  "output": "Compiled with gcc -O2\nok\n",
  "pipelineSteps": ["ms-front-end: parse", "ms-semantic: analyze",
    "ms-codegen-c: generate"],
  "generatedCode": "/* program.c emitido pelo SimpleCCCodeGenerator */"
```

3.11.2 ms-front-end (porta 3001) — POST /parse

Este ativo reutilizável executa a primeira fase do pipeline. Recebe o texto do programa MiniPar 2026.1 e devolve a AST em formato JSON ou uma lista estruturada de erros sintáticos. O contrato permanece independente do back-end de tradução.

```
// Requisição
{ "sourceCode": "class Animal { string name; }" }
// Resposta sem erro
{ "ast": { "type": "Program", "declarations": [ ... ] }, "errors": [] }
// Resposta com erro sintático
{ "ast": null, "errors": [{ "message": "Parser error at 2:11: Expected
LBRACE" }] }
```

3.11.3 ms-semantic (porta 3002) — POST /analyze

O analisador semântico consome exclusivamente a AST serializada produzida na fase anterior. A função `analyze_ast_full` desserializa o documento JSON, percorre a árvore com o visitador `SemanticAnalyzer` e constrói a tabela de símbolos hierárquica. Erros retornam no vetor `errors`, interrompendo o pipeline antes da invocação do back-end.

```
// Requisição: { "ast": { "type": "Program", "declarations": [ ... ] } }
// Resposta:
{
  "ast": { "type": "Program", "declarations": [ ... ] },
  "symbolTable": { "scopes": [ { "name": "global", "symbols": [ ... ] } ] },
  "errors": []
}
```

3.11.4 ms-interpreter (porta 3003) — POST /execute

A variante de execução direta sobre a AST implementa o ponto de adaptação `Interpreter.emit` dentro de um contêiner autônomo. O endpoint recebe a árvore validada, executa o interpretador OO com suporte a blocos `par`, `seq` e canais por socket TCP, e retorna o texto capturado na saída padrão simulada.

```
// Requisição: { "ast": { ... }, "symbolTable": { ... }, "executionMode":
"LOCAL", "target": "INTERPRETER" }
// Resposta: { "output": "woof\n", "exitCode": 0 }
```

3.11.5 ms-codegen-* — POST /generate (portas 3004, 3005, 3007, 3008)

Os back-ends de compilação compartilham o mesmo contrato de entrada: AST validada e tabela de símbolos. O serviço `ms-codegen-c` (3004) produz código C compilado

com gcc -O2. O ms-codegen-rust (3005) emite código Rust e invoca rustc. O ms-codegen-arm (3007) gera assembly ARMv7. O ms-codegen-python (3008) demonstra a extensão da linha de produto sem alteração dos trechos invariantes.

```
// Requisição: { "ast": { ... }, "symbolTable": { ... } }
// Resposta típica (back-end C):
{
  "output": "Compiled with gcc -O2\nok\n",
  "generatedCode": "int main(void) { /* ... */ }",
  "exitCode": 0
}
```

3.11.6 ms-parallel-coord (porta 3006) — POST /coordinate

Quando DISTRIBUTED_SOCKETS é selecionado e o programa-fonte não utiliza canais distribuídos, o gateway encaminha a coordenação a este serviço. Ele dispara, em paralelo, requisições de execução aos três workers socket (portas 9001–9003), cada um executando um programa MiniPar independente. O protocolo de transporte consiste no envio do comando RUN seguido da leitura de um documento JSON.

```
// Resposta:
{
  "output": "=== Coordenador Distribuido ===\n[PC1] QuickSort: ...\n...",
  "results": [
    { "role": "quicksort", "machine": "PC1", "data": "...", "ip":
      "172.x.x.x", "port": 9001 },
    { "role": "matrix", "machine": "PC2", "data": "...", "port": 9002 },
    { "role": "factorial", "machine": "PC3", "data": "...", "port": 9003 }
  ]
}
```

3.11.7 Tratamento de Erros e Encadeamento do Pipeline

O gateway interrompe o encadeamento na primeira fase que retorna erro. Erros sintáticos originam-se em ms-front-end com ast nulo; erros semânticos em ms-semantic com vetor errors não vazio; falhas de compilação propagam-se a partir do campo exitCode distinto de zero no TranslationResult. Cada passo bem-sucedido registra-se em pipelineSteps, permitindo auditoria da execução.

4. Modelagem UML

4.1 Diagrama de Casos de Uso

Os casos de uso principais são: editar código MiniPar, selecionar variante LPS, executar pipeline, visualizar AST e tabela de símbolos, executar paralelismo distribuído (3 máquinas) e visualizar o fractal de Sierpinski.

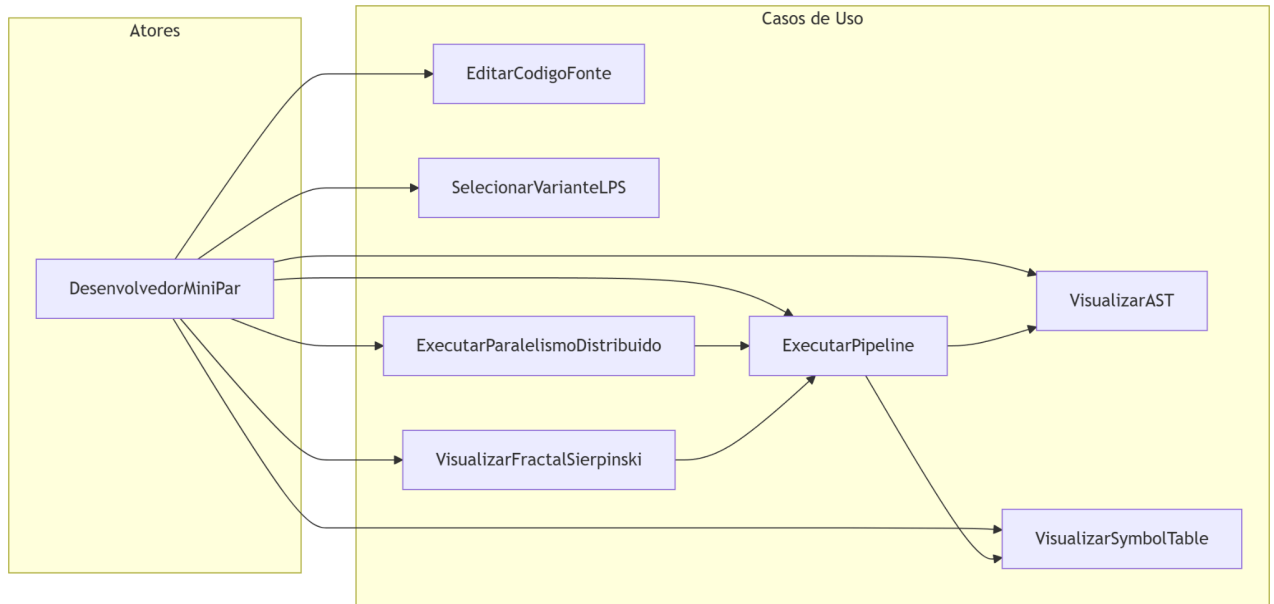


Figura 3: Casos de uso do Desenvolvedor MiniPar com o framework, cobrindo edição de código, seleção de variante LPS, execução da pipeline, visualização de AST e tabela de símbolos, execução do paralelismo distribuído e visualização do fractal Sierpinski.

4.2 Arquitetura de Componentes

Na modelagem por componentes, Léxico, Parser, Analisador Semântico e Geradores aparecem como unidades substituíveis encapsuladas em microsserviços FastAPI, orquestradas pelo api-gateway NestJS.

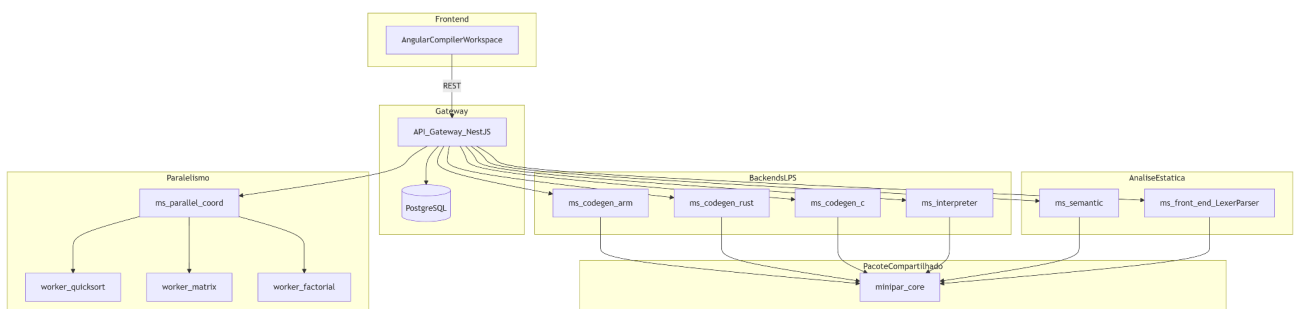


Figura 4: Diagrama de componentes UML mostrando as dependências entre o frontend Angular, o API Gateway, os microsserviços de análise estática, os back-ends LPS, o coordenador de paralelismo e o pacote compartilhado minipar-core.

4.3 Diagrama de Classes

A AST orientada a objetos replica, em JSON, a estrutura dos nós Java de Maciel (2025.1). Exemplo JSON mínimo:

```
{
  "type": "Program",
  "declarations": [{
    "type": "ClassDecl", "name": "Dog", "extends": "Animal",
    "members": [{ "type": "MethodDecl", "name": "bark", ... }]
  }]
}
```

Tabela 3: Nós da AST MiniPar 2026.1 (ast_nodes.py).

Tipo	Campos principais
Program	declarations[] — raiz
ClassDecl	name, extends, members[]
MethodDecl	return_type, name, parameters, body
FuncDecl	Função global procedural
VarDecl	var_type, name, initializer?
Block	statements[]
IfStmt, WhileStmt, ForStmt	Controle de fluxo
ParBlock, SeqBlock	Paralelismo estruturado
NewInstance	class_name, arguments[]
MethodCall	receiver, method, arguments[]
PropertyAccess, ThisExpr, SuperCall	OO
SendStmt, ReceiveStmt, ChannelDecl	Canais IPC
BinaryOp, UnaryOp, literais	Expressões

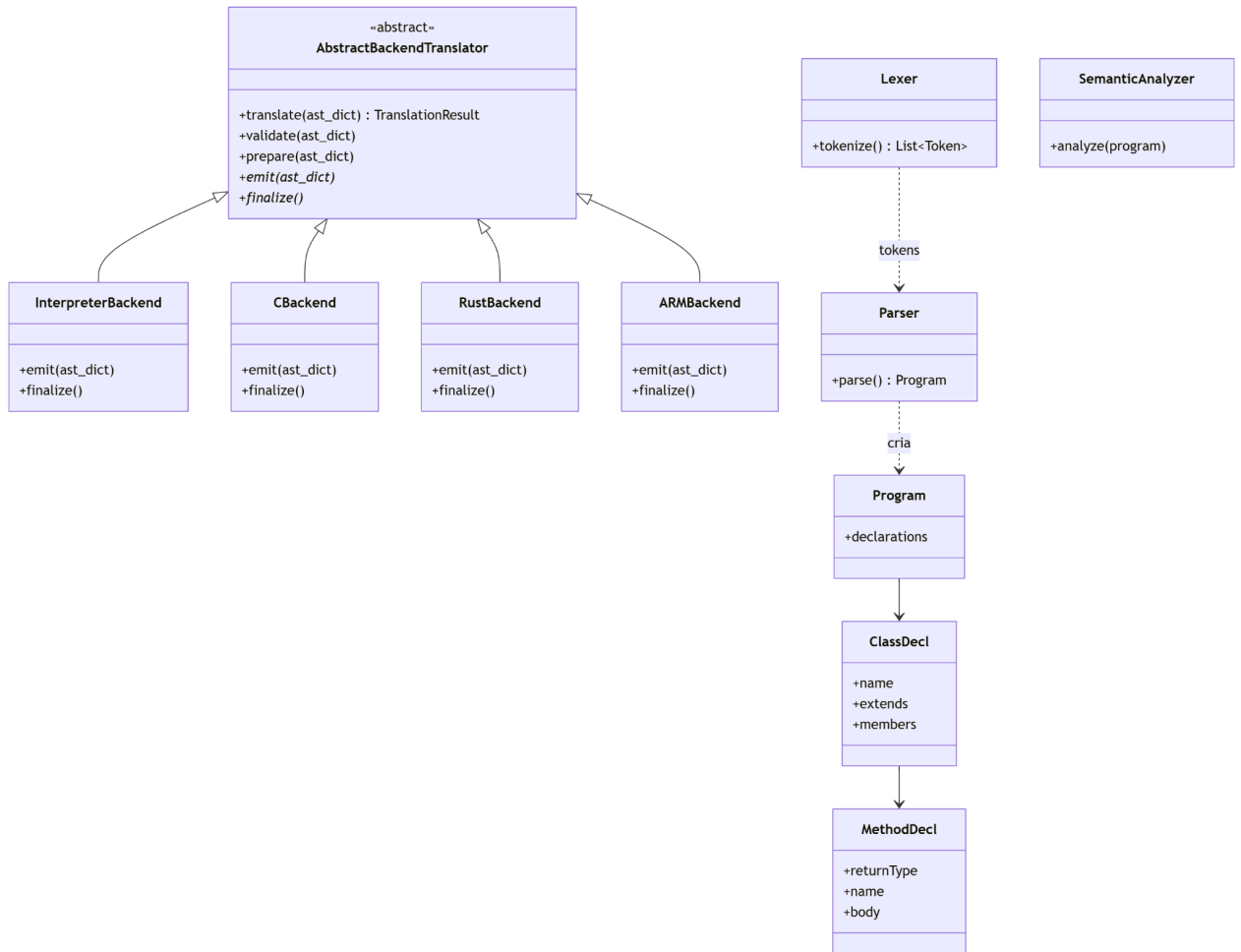


Figura 5: Diagrama de classes do framework, apresentando a hierarquia da classe abstrata *AbstractBackendTranslator* com as subclasses concretas (*InterpreterBackend*, *CBackend*, *RustBackend*, *ARMBBackend*), os nós de AST para suporte a OO (*ClassDecl*, *MethodDecl*) e as dependências entre *Lexer*, *Parser* e *SemanticAnalyzer*.

5. Fases do Compilador MiniPar 2026.1

5.1 Evolução 2025.1 - 2026.1

A MiniPar 2025.1 oferecia interpretador OO em Java (cl-minipar). A versão 2026.1 integra paralelismo par/seq, canais, funções procedurais e cinco back-ends de tradução, reutilizando algoritmos via TAC e gcc.

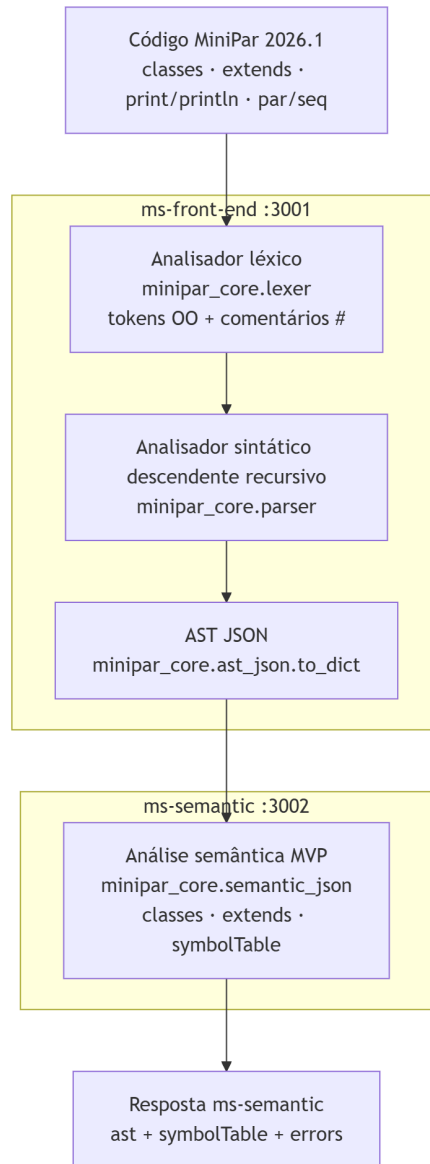


Figura 6: Fluxo interno das fases de análise do compilador, desde a entrada do código MiniPar 2026.1 até a produção da resposta do ms-semantic, passando pelo analisador léxico, pelo parser descendente recursivo e pela análise semântica OO com construção da tabela de símbolos.

5.2 Gramática BNF Completa com Suporte OO

A linguagem MiniPar 2026.1 estende a MiniPar 2025.1 com orientação a objetos completa, paralelismo estruturado e canais de comunicação. Gramática formal implementada em `packages/minipar-core/minipar_core/parser.py`:

```

<program> ::= <declaration>*
<declaration> ::= <class_decl> | <func_decl> | <var_decl>
                | <channel_decl> | <statement>
  
```

```

/* === Declarações OO === */
<class_decl> ::= "class" IDENT ["extends" IDENT] "{" <class_member>* "}"
<class_member> ::= <oo_var_decl> | <method_decl>
<oo_var_decl> ::= <type> IDENT ["=" <expr>] ";"
<method_decl> ::= <type> IDENT "(" [<param_list>] ")" <block>
<param_list> ::= <parameter> ("," <parameter>)*
<parameter> ::= <type> IDENT

/* === Declarações procedurais === */
<func_decl> ::= "func" IDENT "(" [<param_list>] ")" ">" <type> <block>
<var_decl> ::= "var" IDENT ":" <type> ["=" <expr>] ";"
<type> ::= "void"|"number"|"string"|"bool"|"list"|"dict"|"any"|IDENT

/* === Canais === */
<channel_decl> ::= ("s_channel"|"c_channel") IDENT ["{" <expr_list> "}"] ";"
<send_stmt> ::= "send" "(" IDENT "," <expr> ")" ";"
<receive_stmt> ::= "receive" "(" IDENT "," IDENT ")" ";"

/* === Paralelismo === */
<par_block> ::= "par" "{" <statement>+ "}"
<seq_block> ::= "seq" "{" <statement>+ "}"

/* === Statements === */
<statement> ::= <expr_stmt> | <block> | <if_stmt> | <while_stmt>
               | <for_stmt> | <return_stmt> | <var_decl>
               | <send_stmt> | <receive_stmt> | <par_block> | <seq_block>

/* === Expressões (precedência decrescente) === */
<expr> ::= <assignment>
<assignment> ::= <conditional> ["=" <assignment>] | <postfix> "="
<assignment>
<conditional> ::= <logical_or> ["?" <expr> ":" <expr>]
<logical_or> ::= <logical_and> ("||" <logical_and>)*
<logical_and> ::= <equality> ("&&" <equality>)*
<equality> ::= <relational> (("=="|"!=") <relational>)*
<relational> ::= <additive> ((" "<"|<="|>"|>=") <additive>)*
<additive> ::= <multiplicative> (("+"|"-" <multiplicative>)*
<multiplicative> ::= <unary> (("*"|"/"|"%" <unary>)*
<unary> ::= ("!"|"-"|"+" <unary> | <postfix>

/* === Acesso a membros e indexação === */
<postfix> ::= <primary> (<call> | <member_access> | <index>)*
<primary> ::= IDENT | NUMBER | STRING | BOOL | "(" <expr> ")" | <new_expr>
<new_expr> ::= "new" IDENT "(" [<expr_list>] ")"
<call> ::= "(" [<expr_list>] ")"
<member_access> ::= "." IDENT [<call>]
<index> ::= "[" <expr> "]" ["=" <expr>]
<expr_list> ::= <expr> ("," <expr>)*

```

5.2.1 Palavras-chave e Tabela de Tokens

Categoria	Tokens / palavras-chave	Implementação
Tipos primitivos	void, number, string, bool, list, dict, any	KEYWORDS
OO	class, extends, new, this, super	read_identifier()
Controle	if, else, while, for, break, continue, return	idem
Paralelismo	par, seq	blocos estruturados
Canais	s_channel, c_channel, send, receive	MVP Fase 1–2
Procedural	func, var	declarações globais
E/S	println, print	PrintStmt
Literais	NUMBER_LITERAL, STRING_LITERAL, true/false	read_number/string
Operadores	PLUS..OR, ASSIGN, DOT	scanner principal
Delimitadores	LPAREN..RBRACKET, SEMICOLON, ARROW	tokenize()

5.2.2 Exemplos de Código Válido

Exemplo 1 — Classe com herança:

```
class Animal { string name; }  
  
class Dog extends Animal {  
    void bark() { println("woof"); }  
}  
  
class Main {  
    void run() { Dog d = new Dog(); d.bark(); }  
}
```

Exemplo 2 — Paralelismo estruturado:

```
class Main {  
    void run() {  
        seq { println("Task 1"); }  
        par { println("Task 2a"); println("Task 2b"); }  
    }  
}
```

```
}
```

Exemplo 3 — Recursão e condicional:

```
class Factorial {  
    number compute(number n) {  
        if (n <= 1) { return 1; }  
        else { return n * this.compute(n - 1); }  
    }  
}
```

5.2.3 Restrições Semânticas

A análise semântica percorre declarações de classe, constrói `symbolTable.scopes[]` e acumula erros sem abortar o pipeline:

```
# Classe duplicada  
if name in known_classes:  
    errors.append(f"Semantic error: Class '{name}' already declared")  
  
# Membro duplicado em classe  
if mname in seen:  
    errors.append(f"Semantic error: Duplicate member '{mname}' in class  
'{name}''")  
  
# Herança inválida  
if extends and extends not in known_classes:  
    errors.append(f"Semantic error: Superclass '{extends}' not found for  
class '{name}''")
```

Regras adicionais implementadas: toda classe com `extends` deve referenciar superclasse previamente declarada, verificada após o registro de todas as classes; a proibição de membros duplicados no mesmo escopo de classe; a execução concorrente dos comandos em blocos par mediante processos e broker TCP no interpretador, com uso de `c_channel` no modo distribuído; e a comunicação por canais `s_channel` e `c_channel` com operações `send` e `receive` via `socket_channel.py`, validada pelo programa `15_channels.minipar`, cuja saída esperada é o valor 42. A verificação de tipos permanece em nível mínimo viável, admitindo `upcast` implícito em herança.

5.3 Pseudocódigos e Implementação dos Analisadores

5.3.1 Analisador Léxico

O módulo `minipar_core.lexer` implementa a classe `Lexer`, cujo método `tokenize()` percorre o código-fonte aplicando `skip_whitespace()`, `skip_comment()` (`#` e `/* */`) e os leitores especializados:

```
def tokenize(self) -> List[Token]:
    while self.pos < len(self.source):
        self.skip_whitespace()
        if self.skip_comment(): continue
        if char.isdigit(): self.tokens.append(self.read_number())
        elif char == '"': self.tokens.append(self.read_string())
        elif char.isalpha() or char == '_':
            tok = self.read_identifier()
            if tok.value in KEYWORDS: tok.type = KEYWORDS[tok.value]
            self.tokens.append(tok)
        else: self.tokens.append(self.read_operator())
        self.tokens.append(Token(TokenType.EOF, None, self.line, self.column))
    return self.tokens
```

Pseudocódigo equivalente em linguagem algorítmica:

```
funcao tokenize(fonte):
    posicao <- 0
    enquanto posicao < comprimento(fonte):
        ignorar espacos em branco
        se comentario detectado entao avancar e continuar
        se digito entao emitir NUMBER_LITERAL
        senao se aspas duplas entao emitir STRING_LITERAL
        senao se letra ou sublinhado entao
            lexema <- ler identificador ou palavra-chave
            emitir tipo correspondente em KEYWORDS ou IDENTIFIER
        senao emitir operador ou delimitador de tabela fixa
        emitir EOF; retornar lista de tokens
```

5.3.2 Analisador Sintático e AST

O Parser descendente recursivo consome tokens e constrói Program. A produção de classe OO:

```

def class_declaration(self) -> ClassDecl:
    self.consume(TokenType.CLASS)

    name = self.consume(TokenType.IDENTIFIER, "Expected class name").value
    extends = None

    if self.match(TokenType.EXTENDS):
        self.advance()

    extends = self.consume(TokenType.IDENTIFIER, "Expected superclass
name").value

    self.consume(TokenType.LBRACE)
    members = []
    while not self.match(TokenType.RBRACE):
        if self.is_oo_var_start(): members.append(self.oo_var_declaration())
        elif self.is_method_start(): members.append(self.method_declaration())
    return ClassDecl(name=name, extends=extends, members=members)

```

Pseudocódigo de expressões com precedência:

```

funcao parse_assignment():
    no <- parse_conditional()
    se token atual for "=" entao
        consumir "="
        direita <- parse_assignment()
        retornar no de atribuicao
    retornar no

funcao parse_additive():
    esquerda <- parse_multiplicative()
    enquanto token em {+, -}:
        operador <- consumir operador
        direita <- parse_multiplicative()
    esquerda <- no binario(esquerda, operador, direita)
    retornar esquerda

```

5.3.3 Coordenador de Paralelismo Distribuído (ms-parallel-coord)

```

para cada host em [quicksort, matrix, factorial]:
    disparar thread dispatch_worker(host, port)
aguardar todas as threads (join)
ordenar resultados por role (PC1, PC2, PC3)

```

```
montar output agregado com rotulos PC1, PC2, PC3
retornar { output, results[] }
```

5.3.4 Worker Socket (processo independente)

```
servidor escuta na porta configurada (9001--9003)
ao receber conexao:
    ler comando "RUN"
    executar algoritmo (QuickSort | Matriz | Fatorial)
    enviar JSON { role, machine, data } via socket
    fechar conexao
```

5.3.5 Analisador Semântico e Tabela de Símbolos

O microserviço ms-semantic invoca `semantic_full.analyze_ast_full`: desserializa AST JSON, executa `SemanticAnalyzer` e retorna `symbolTable.scopes[]` e erros. Pseudocódigo do visitador semântico:

```
funcao visitar_programa(no):
    para cada declaracao em no.declarations: visitar(declaracao)
    para cada classe com extends:
        se superclasse nao registrada entao registrar erro semantico

funcao visitar_classe(no):
    se no.nome ja existe entao registrar erro de duplicata
    inserir simbolo de classe no escopo global
    abrir escopo de classe
    para cada membro em no.members:
        se nome de membro repetido entao registrar erro
        visitar(membro)
    fechar escopo de classe
```

5.3.6 Emissão de Código C e Invocação do Compilador Externo

Pseudocódigo do back-end C, incluindo `emit()` e `finalize()`:

```
funcao emit(ast_serializada):
    tac <- TACGenerator.lower(ast_serializada)
    codigo_c <- SimpleCCCodeGenerator.generate(tac)
    armazenar codigo_c internamente
```



```

funcao finalize():
    arquivo <- gravar temporario(codigo_c)
    resultado_compilacao <- executar(["gcc", "-O2", "-lm", arquivo])
    se codigo de retorno diferente de zero entao
        retornar TranslationResult com mensagem de erro
    saida <- executar(binario gerado)
    retornar TranslationResult(saida, codigo_c, codigo de retorno)

```

5.4 Representação Intermediária TAC

O TACGenerator (tac_codegen.py) realiza lowering AST → lista de instruções de três endereços (TAC):

Tabela 5: Operações TAC principais (tac.py).

Operação	Semântica
ASSIGN, aritméticos, UNARY	Expressões e temporários t0, t1...
IF_FALSE, IF_TRUE, GOTO, LABEL	Controle de fluxo
FUNC_BEGIN, FUNC_END, FUNC_PARAM	Procedimentos e métodos OO
CALL, RETURN, CALL_MAIN	Chamadas e ponto de entrada
PRINT	Saída (println)
PAR_BEGIN, PAR_END, SEQ_BEGIN, SEQ_END	Paralelismo
THREAD_START, THREAD_END	Threads (interpretador)

Lowering de método OO em TAC:

```

def gen_MethodDecl(self, node: MethodDecl) -> None:
    class_name = self.class_stack[-1]
    fn_name = f"{class_name}_{node.name}"
    self.emit("FUNC_BEGIN", fn_name)
    for param in node.parameters:
        self.emit("FUNC_PARAM", param.name)
    self.generate(node.body)
    self.emit("FUNC_END", fn_name)

```

5.5 Back-ends de Tradução

Cada back-end implementa hotspots emit/finalize sem alterar translate():

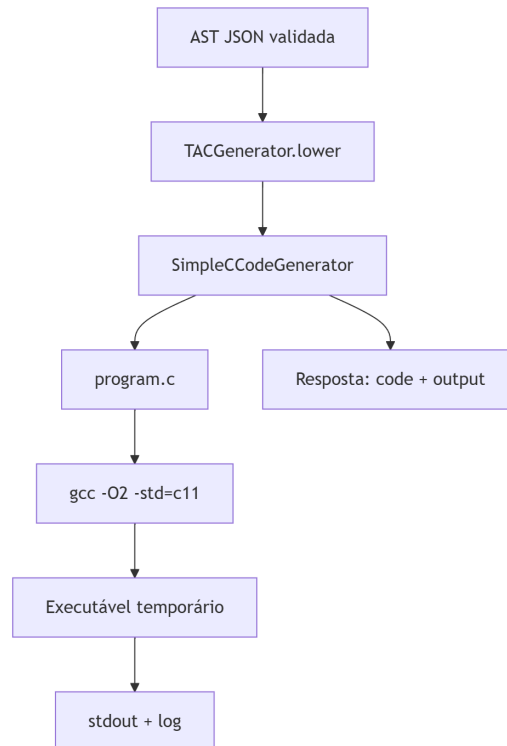


Figura 7: Fluxo de geração de código do back-end C, desde a AST JSON validada até a produção do executável, passando pelo TACGenerator, pelo SimpleCCodeGenerator, pela escrita do arquivo program.c e pela compilação com gcc -O2 -std=c11.

5.5.1 Interpretador

Executa AST diretamente via from_dict() — sem TAC. Suporta OO (new, herança, this), par/seq com threads e E/S via StringIO.

```
def emit(self, ast_dict: dict) -> None:
    program = from_dict(ast_dict)
    try:
        self._exec_output = self.execute(program)
    except RuntimeError as exc:
        self._errors.append(str(exc))
```

5.5.2 Back-end C (CBackend)

Pipeline TAC → SimpleCCodeGenerator → arquivo temporário → gcc -O2 -lm:

```
def emit(self, ast_dict: dict) -> None:
    tac = TACGenerator().lower(ast_dict)
    self._code = SimpleCCodeGenerator().generate(tac)

def _compile_and_run(self, c_code: str) -> dict:
```

```
compile_cmd = [self.compiler, "-O2", self.std_flag, c_file, "-o", exe_file, "-lm"]
comp = subprocess.run(compile_cmd, capture_output=True, text=True)
run = subprocess.run([str(exe_file)], capture_output=True, text=True, timeout=30)
```

5.5.3 Back-end C++ (CppBackend)

Herda CBackend; altera apenas `compiler = "g++"` e `std_flag = "-std=c++17"` — reuso por herança (Gamma).

5.5.4 Back-end Rust (RustBackend)

Emissão direta AST → Rust via RustASTEmitter (sem TAC no MVP). Invoca `rustc` quando disponível no contêiner.

5.5.5 Back-end ARM (ARMBackend)

TAC → SimpleARMGenerator (ARMv7). Retorna `assembly` em `TranslationResult.code`; mensagem explícita se `toolchain arm-linux-gnueabi-hf-gcc` ausente.

5.5.6 Paralelismo par/seq e comunicação por soquetes

Blocos `seq` executam `statements` em ordem; blocos `par` disparam processos filhos com comunicação interprocessos mediada por `sockets TCP` locais, sem memória compartilhada. O módulo `socket_channel.py` implementa um broker para canais `s_channel` e `c_channel`. O programa `15_channels.minipar` valida esse mecanismo com a passagem do inteiro 42 e impressão do mesmo valor na saída padrão.

No modo `DISTRIBUTED_SOCKETS`, o serviço `ms-parallel-coord` estabelece conexões `TCP` paralelas aos três `workers` (portas 9001–9003); cada `worker` recebe o comando `RUN`, executa um programa `MiniPar` fixo e devolve documento `JSON` com endereço `IP`, porta e resultado computado. Limitações honestas: modelo de `threads` simplificado; `send/receive` com suporte parcial (MVP).

6. Técnicas de Reuso

6.1 Engenharia de Domínio Aplicada a Compiladores

O domínio "pipeline MiniPar" foi decomposto em subsistemas com contratos REST estáveis. Artefatos reutilizáveis incluem lexer, parser, semântica, TAC e esqueleto de tradução; variabilidade concentra-se nos hotspots de emit/finalize e na configuração LPS.

6.2 Padrão Template Method: Análise Profunda

A variabilidade entre back-ends não usa ramificações dispersas no gateway, e sim o padrão Template Method. Frozen-spots (translate, validate, prepare) garantem ordem e tratamento uniforme de erros; hotspots (emit, finalize, hooks) encapsulam algoritmos específicos. Variantes concretas: Interpreter (semântica dinâmica), CBackend/CppBackend (TAC + gcc/g++), RustBackend (AST → Rust), ARMBackend (lowering TAC → ARMv7).

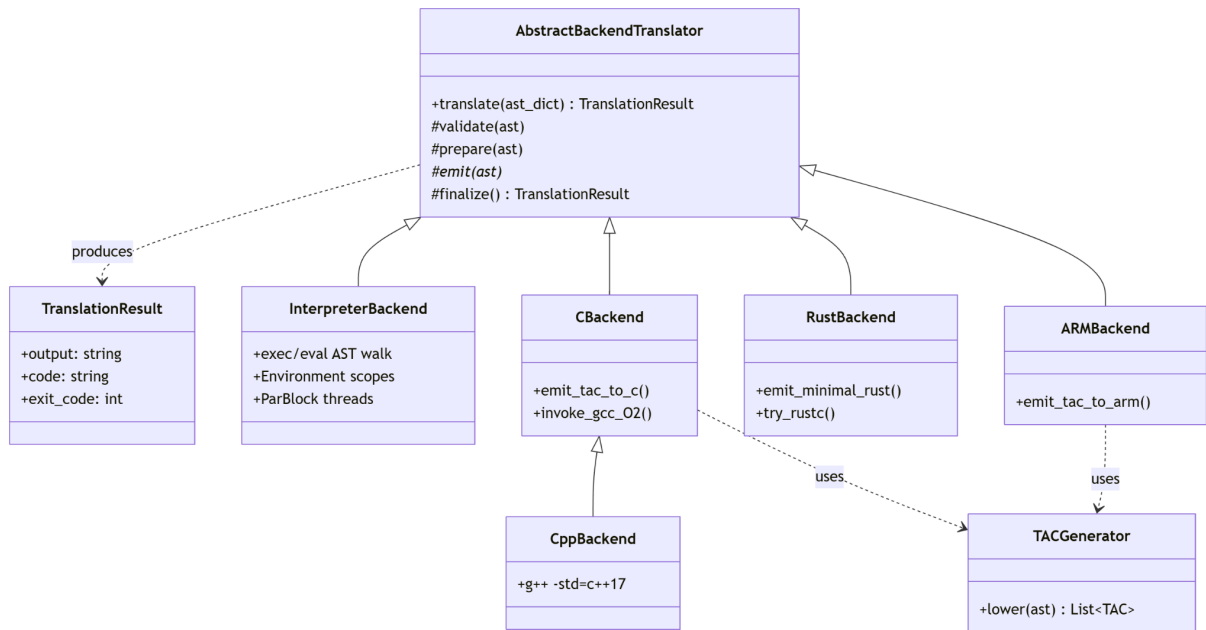


Figura 8: Diagrama de classes do padrão Template Method aplicado ao minipar-core, com a classe abstrata *AbstractBackendTranslator* definindo os métodos *validate*, *prepare*, *emit* e *finalize*, e as subclasses concretas *CBackend*, *CppBackend*, *RustBackend*, *ARMBackend* e *InterpreterBackend* implementando os hotspots de cada variante.

Esqueleto `AbstractBackendTranslator.translate()`:

```
def translate(self, ast_dict):
    self.validate(ast_dict)
    if self._errors:
        return TranslationResult(..., errors=self._errors)
```

```

self.prepare(ast_dict)
self.emit(ast_dict) # hotspot por back-end
return self.finalize()

```

6.3 Mapa de Reuso 2025.1 - 2026.1

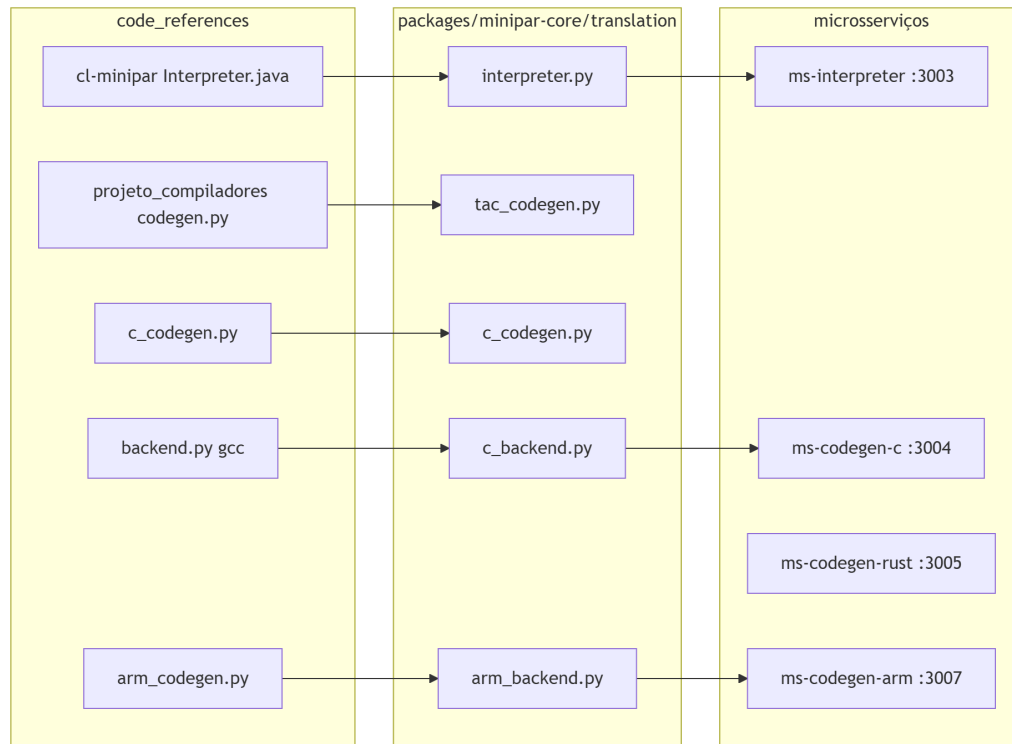


Figura 9: Mapa de reuso sistemático do projeto, relacionando os componentes originais de code_references (Interpreter.java, codegen.py, c_codegen.py, arm_codegen.py, backend.py) com os módulos correspondentes do minipar-core e os microsserviços que os consomem.

Tabela 6: Componentes reutilizados e destino no framework.

Origem (2025.1)	Artefato	Destino 2026.1
Maciel	Lexer, Parser, AST OO, Interpreter	lexer.py, parser.py, interpreter.py
Rego	TAC, gerador C, ARM, integração gcc	tac_codegen.py, c_backend.py, arm_backend.py
Rego / procedural	Análise semântica base	semantic.py, semantic_json.py
Equipe 2026.1	Gateway, Docker, MS REST	api-gateway/, microservices/

6.4 Componentes, Acoplamento e Contratos

Microserviços comunicam-se exclusivamente via JSON (`_AST_CONTRACT.md`). Acoplamento reduzido: `ms-semantic` não conhece back-end alvo; `ms-codegen-c` não reimplementa parser. Contrato `TranslationResult` unifica saída (`output`, `code`, `exit_code`).

6.5 Registry (OCP) e Strategy Implícita

`BACKEND_REGISTRY` implementa `Registry` + `Strategy`: o pipeline seleciona implementação por `targetVariability` sem modificar código de orquestração. Adicionar variante = nova entrada + MS — alinhado ao Open-Closed Principle.

6.6 Padrões Adicionais Identificados

- Template Method — tradução (`AbstractBackendTranslator`)
- Registry — `backend-registry.ts`
- Strategy — back-ends intercambiáveis via mesma interface
- Facade — `PipelineService` oculta detalhes dos microserviços
- Adapter — `ast_from_json.from_dict` converte AST dict → objetos Python

6.7 Métricas Narrativas de Reuso

Estimativa qualitativa do monorepo: ~70% do código de análise (lexer, parser, semântica, AST, TAC) é compartilhado entre todas as instâncias; ~15% corresponde a hotspots por back-end (`emit/finalize`); ~15% a infraestrutura MS/gateway/frontend. Seis variantes (cinco de referência + extensão Python) reutilizam o mesmo núcleo sem duplicação de parser.

6.8 Hooks Opcionais (`hook_validate`, `hook_prepare`)

Além dos métodos abstratos `emit()/finalize()`, a classe base expõe hooks opcionais invocados dentro dos frozen-spots `validate()` e `prepare()`. Se não sobrescritos, comportam-se como no-op:

```
def hook_validate(self, ast_dict: dict) -> None:
    if not ast_dict.get("declarations"):
        self._errors.append("Programa vazio: nenhuma declaração encontrada")

def hook_prepare(self, ast_dict: dict) -> None:
```

```
pass # pré-processamento opcional antes de emit()
```

6.9 Hotspots Futuros (Roadmap)

Conforme orientação do Prof. Arturo: formalizar contratos substituíveis para lexer (manual versus autômatos DFA/NFA) e parser (top-down recursivo versus bottom-up LR/LALR).

6.10 Contrato `_template_backend.py`

Artefato `implementado` em `packages/minipar-core/minipar_core/translation/_template_backend.py` — esqueleto copiável com hotspots vazios e hooks opcionais documentados:

```
class TemplateBackend(AbstractBackendTranslator):
    def emit(self, ast_dict: dict) -> None:
        raise NotImplementedError("Implemente emit(): TAC, AST direta, bytecode...")

    def finalize(self) -> TranslationResult:
        raise NotImplementedError("Implemente finalize(): retorne TranslationResult")
```

7. Implementação

7.1 Estrutura do Monorepo

O projeto `minipar-framework` organiza todas as aplicações em um monorepo com pastas de responsabilidade bem definidas:

- A pasta `frontend` contém a aplicação Angular com o editor de código, o seletor de variabilidade e o painel de resultados.
- A pasta `api-gateway` contém o serviço Node.js responsável pela orquestração do pipeline e pela persistência no banco de dados.
- A pasta `database` contém o arquivo `init.sql` com o schema PostgreSQL, incluindo a tabela `compilation_history` com os campos `id`, `source_code`, `target_variability`, `execution_mode`, `status`, `output` e `created_at`.
- A pasta `microservices` contém um subdiretório para cada microsserviço, cada um com seu próprio README de especificação, Dockerfile e código-fonte.

- A pasta docs/diagrams centraliza todos os diagramas Mermaid (.mmd) do projeto. A raiz do repositório contém o docker-compose.yml principal, o PROJECT_REQUIREMENTS.md com a especificação acadêmica.

7.2 Tecnologias Utilizadas

A interface web foi desenvolvida em Angular com TypeScript, oferecendo um editor de código com realce de sintaxe, dropdowns para seleção de variabilidade e modo de execução, e painel de saída com abas para AST, tabela de símbolos, erros semânticos e código gerado. O gateway de API foi implementado em Node.js e expõe os endpoints POST /api/v1/process para processamento de código e GET /health para verificação de disponibilidade. Os microserviços de análise e geração de código foram implementados em Python com FastAPI, permitindo integração nativa com as bibliotecas de compilação reutilizadas da versão 2025.1. O banco de dados PostgreSQL persiste o histórico de compilações e é acessado pelo gateway via ORM. O Docker Compose orquestra todos os serviços com healthchecks e variáveis de ambiente para configuração de modo de pipeline (mock ou http) e URLs dos microserviços.

7.3 API Gateway e Microserviços

O API Gateway centraliza as requisições do frontend em POST /api/v1/process, orquestra a pipeline sequencialmente e persiste o histórico no PostgreSQL. A tabela abaixo resume cada microserviço:

Serviço	Porta	Endpoint	Fase	Responsabilidade
ms-front-end	3001	POST/parse	1	Análise léxica e sintática; retorna AST JSON
ms-semantic	3002	POST/analyze	1	Verificação de tipos, escopos e herança OO; retorna AST anotada e tabela de símbolos.
ms-interpretter	3003	POST/execute	2	Execução direta da AST; OO, seq/par
ms-codegen-c	3004	POST/generate	2	Código C ou C++ + gcc -O2

ms-codegen-rust	3005	POST/generate	2	Código Rust + rustc
ms-codegen-arm	3007	POST/generate	2	Geração de Assembly ARMv7
ms-codegen-python	3008	POST /generate	ext.	Código Python + python3
ms-parallel-coord	3006	POST/coordinate		Coordenação distribuída via sockets (QuickSort, Matrizes, Fatorial)
worker-quicksort	9001	socket	3	QuickSort (PC1)
worker-matrix	9002	socket	3	Multiplicação de matrizes (PC2)
worker-factorial	9003	socket	3	Fatorial (PC3)

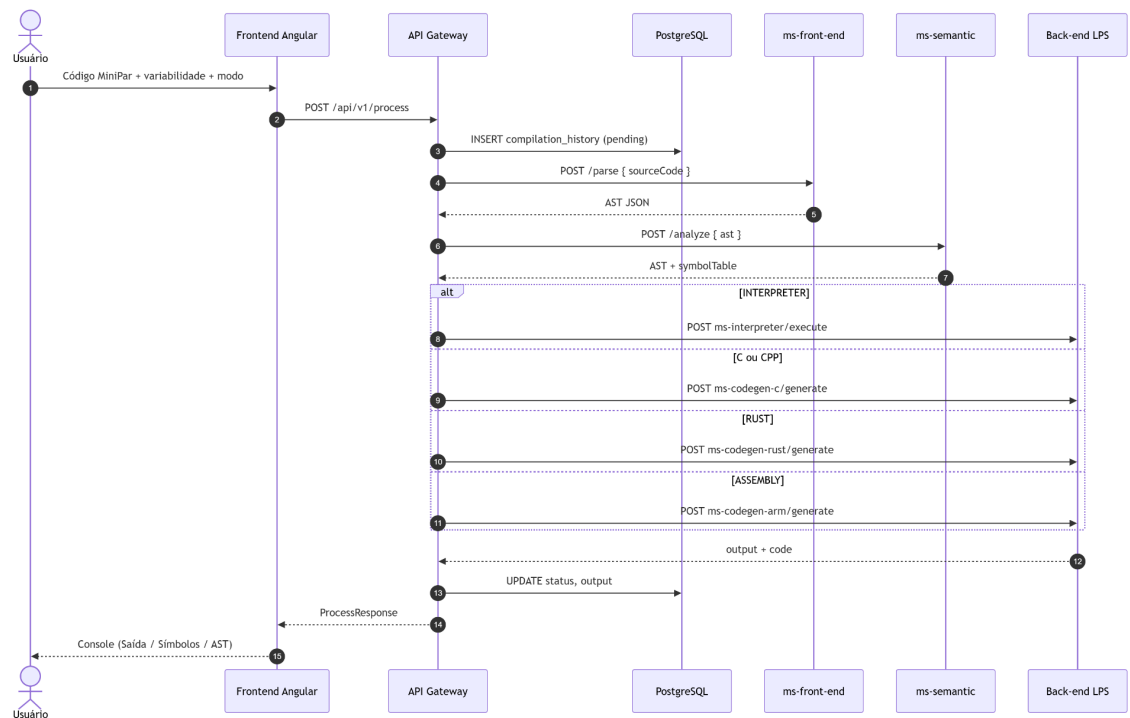


Figura 10: Diagrama de sequência do endpoint `POST /api/v1/process`, detalhando o fluxo desde a submissão do código pelo usuário até o retorno do resultado, passando pelas etapas de parsing, análise semântica, roteamento para o back-end LPS selecionado e persistência no PostgreSQL.

7.4 Feature Model e Pontos de Variação

A LPS MiniPar 2026.1 articula três eixos ortogonais: (1) modo interpretador versus compilador; (2) back-end de código (C, C++, Rust, ARMv7); (3) ambiente local versus distribuído. Cada feature mapeia para assets reutilizáveis (microserviços). (Figura: Árvore de features LPS — modo, back-end, ambiente.)

7.5 Binding Time

Tabela 8: Binding time das features LPS.

Momento	Mecanismo	Exemplo
Implement time	Hotspots em translation/	PythonBackend.emit()
Deploy time	docker-compose.yml, env MS_*	Habilitar ms-codegen-python:3008
Runtime / request time	POST /api/v1/process	targetVariability: "C"
Runtime / request time	feature-panel (UI)	Modo DISTRIBUTED_SOCKETS

7.6 Matriz de Configurações de Produto

Tabela 9: Configurações de produto válidas.

Produto	targetVariability	executionMode	Microserviços ativos
Interpretador local	INTERPRETER	LOCAL	front-end, semantic, interpreter
Compilador C local	C	LOCAL	front-end, semantic, codegen-c
Compilador C++ local	CPP	LOCAL	front-end, semantic, codegen-c
Rust MVP	RUST	LOCAL	front-end, semantic, codegen-rust
ARM MVP	ASSEMBLY	LOCAL	front-end, semantic, codegen-arm
Interpretador distribuído	INTERPRETER	DISTRIBUTED_SOCKETS	+ parallel-coord + workers
Python (extensão)	PYTHON	LOCAL	+ codegen-python

7.7 Microserviços como Assets Reutilizáveis

Cada MS é um asset LPS: ms-front-end e ms-semantic compõem todos os produtos; back-ends são plugáveis. Novo produto = subset do Compose + registry — não fork do repositório.

7.8 Docker Compose como Configuração de Produto

O arquivo docker-compose.yml materializa variabilidade em deploy time: serviços, portas 3000–3008 e 9001–9003, PIPELINE_MODE=http, PIPELINE_BACKEND_MODE=http. Alterar produto ativo equivale a habilitar/desabilitar serviços e env vars — padrão FeatureIDE em infraestrutura.

7.9 Decisão LPS: Microserviços versus FeatureIDE

Optou-se por LPS implementada em código + infraestrutura porque: (i) demonstração E2E real com Docker; (ii) contratos REST alinhados à disciplina de microserviços; (iii) binding runtime via API; (iv) FeatureIDE permanece documentado na árvore de features como modelo conceitual.

7.10 Decisão Arquitetural: NestJS versus Spring Boot

O gateway usa NestJS (TypeScript) por alinhamento com frontend Angular, tipagem compartilhada de enums (TargetVariability), ecossistema leve para MVP acadêmico e hot-reload em desenvolvimento. Spring Boot permaneceria válido para LPS enterprise, porém adicionaria camada JVM separada do núcleo Python — decisão registrada em ROADMAP.md.

8. Testes e Resultados

8.1 Resultados da Fase 1

Validou-se o pipeline pela interface web (ação Executar, atalhos Ctrl+Enter e F5) e por requisições diretas ao gateway:

```
curl -X POST http://localhost:3000/api/v1/process \
-H "Content-Type: application/json" \
-d '{"sourceCode":"class Main { void run() { println(\"ok\"); } }",
  "targetVariability":"INTERPRETER","executionMode":"LOCAL"}'
```

A resposta contém pipelineSteps com ms-front-end: parse e ms-semantic: analyze (sem sufixo mock), além de nós ClassDecl na AST serializada.

8.2 Casos de validação manual

Os fixtures em sources/examples/ cobrem erros sintáticos, semânticos e casos válidos:

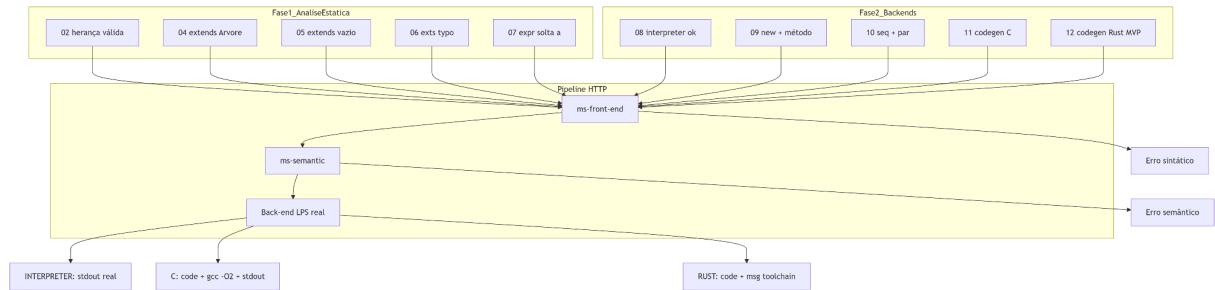


Figura 11: Diagrama de sequência do teste de paralelismo real em três máquinas, mostrando o fluxo completo desde a requisição com `executionMode DISTRIBUTED_SOCKETS` até a agregação dos resultados do QuickSort, da multiplicação de matrizes e do fatorial pelos workers via sockets TCP.

Tabela 10: Casos de validação manual — Fase 1.

Caso	Arquivo	Resultado
Classe com herança válida	02_class_extends	success: true
extends superclasse inexistente	04_semantic_extends_unknown	Erro semântico
extends sem nome	05_parse_extends_missing	Expected superclass name
Palavra-chave inválida	06_parse_invalid_keyword	Expected LBACE
Expressão solta	07_expr_only	ExprStmt (análise OK)

8.3 Casos de validação — Fase 2 (back-ends)

Mensagens representativas registradas na Fase 1:

- Semantic error: Superclass 'Arvore' not found for class 'Dog'
- Parser error at 5:20: Expected superclass name
- Parser error at 5:11: Expected LBACE

Erros de análise aparecem no painel Console da UI (chip de status e mensagem destacada); execuções bem-sucedidas habilitam as abas Saída, Símbolos e AST.

Tabela 11: Casos de validação — Fase 2 (back-ends).

Arquivo	LPS	Resultado	Pipeline step
08_interpreter_ok	INTERPRETER	Saída ok	ms-interpreter: execute
09_oo_new	INTERPRETER	Saída woof	ms-interpreter: execute
10_par_seq	INTERPRETER	Saída a b c	ms-interpreter: execute
11_codegen_c	C	C + gcc -O2 + stdout	ms-codegen-c: generate
12_codegen_rust_stub	RUST	Código Rust MVP	ms-codegen-rust: generate
16_codegen_python	PYTHON	hello from Python backend	ms-codegen-python: generate

8.4 Resultados da Fase 2 — Back-ends reais

Com PIPELINE_BACKEND_MODE=http, interpretador e geradores passaram a responder com saída produzida pelo núcleo minipar-core:

```
# targetVariability: INTERPRETER
# output: "ok\n"
class Main { void run() { println("ok"); } }
```

```
# targetVariability: C
curl -X POST http://localhost:3000/api/v1/process \
-H "Content-Type: application/json" \
-d '{"sourceCode":"class Main { void run() { println(\"ok\"); } }",
  "targetVariability":"C","executionMode":"LOCAL"}'
# pipelineSteps: ms-front-end: parse, ms-semantic: analyze, ms-codegen-c:
generate
# output: "Compiled with gcc -O2\nok\n"
# generatedCode: program.c gerado
```

Os arquivos 08_interpreter_ok a 16_codegen_python documentam o comportamento observado. As variantes Rust e ARM permanecem em nível MVP: geram código sintaticamente válido e retornam mensagem explícita quando rustc ou a toolchain ARM não estão disponíveis no contêiner.

8.5 Evidências Visuais da Interface Web

Capturas de tela da UI Angular (frontend/), obtidas com scripts/capture-ui-screenshots.mj(Playwright) contra ng serve e stack Docker local. Complementam os diagramas de sequência com evidência da execução real na LPS.

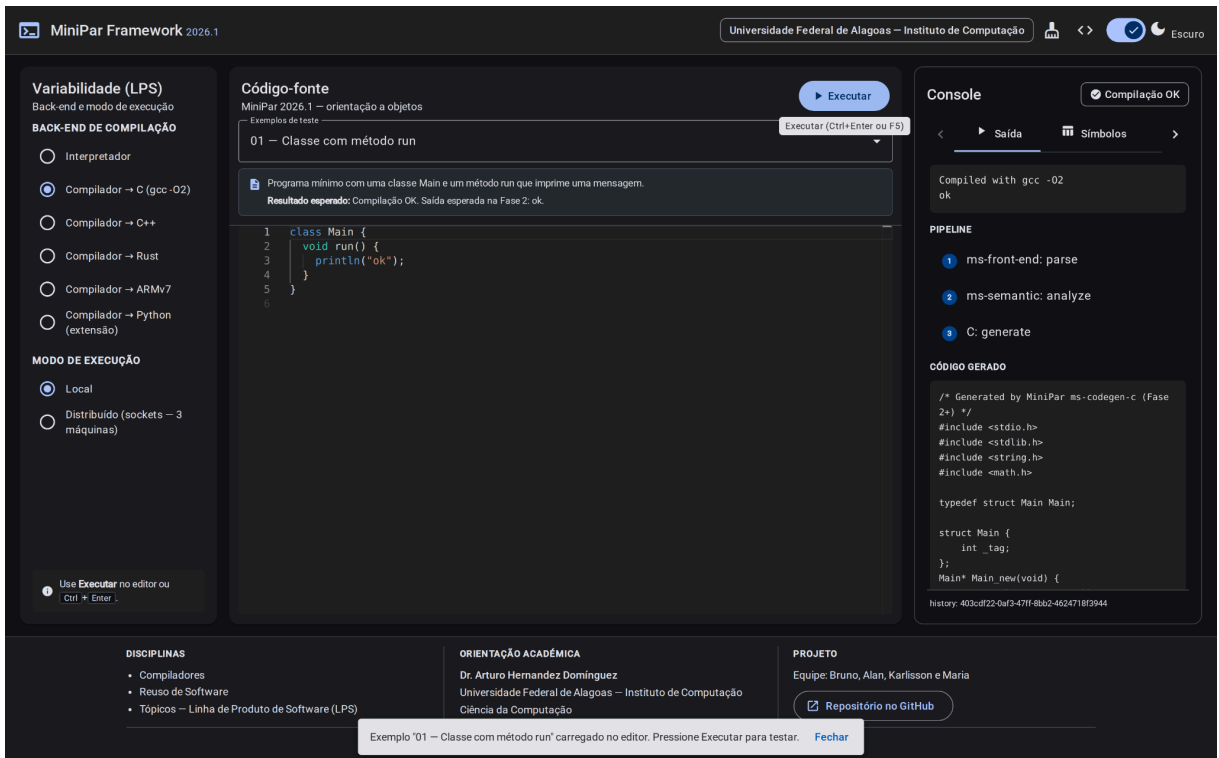
8.6 Teste de Real Paralelismo (3 Computadores)

Implementou-se o microserviço ms-parallel-coord (:3006) e três workers Python como processos independentes comunicando-se via sockets TCP (:9001–9003). O modo DISTRIBUTED_SOCKETS na UI dispara o coordenador após a análise semântica; os resultados agregados aparecem no Console e no campo distributedResults da resposta JSON.

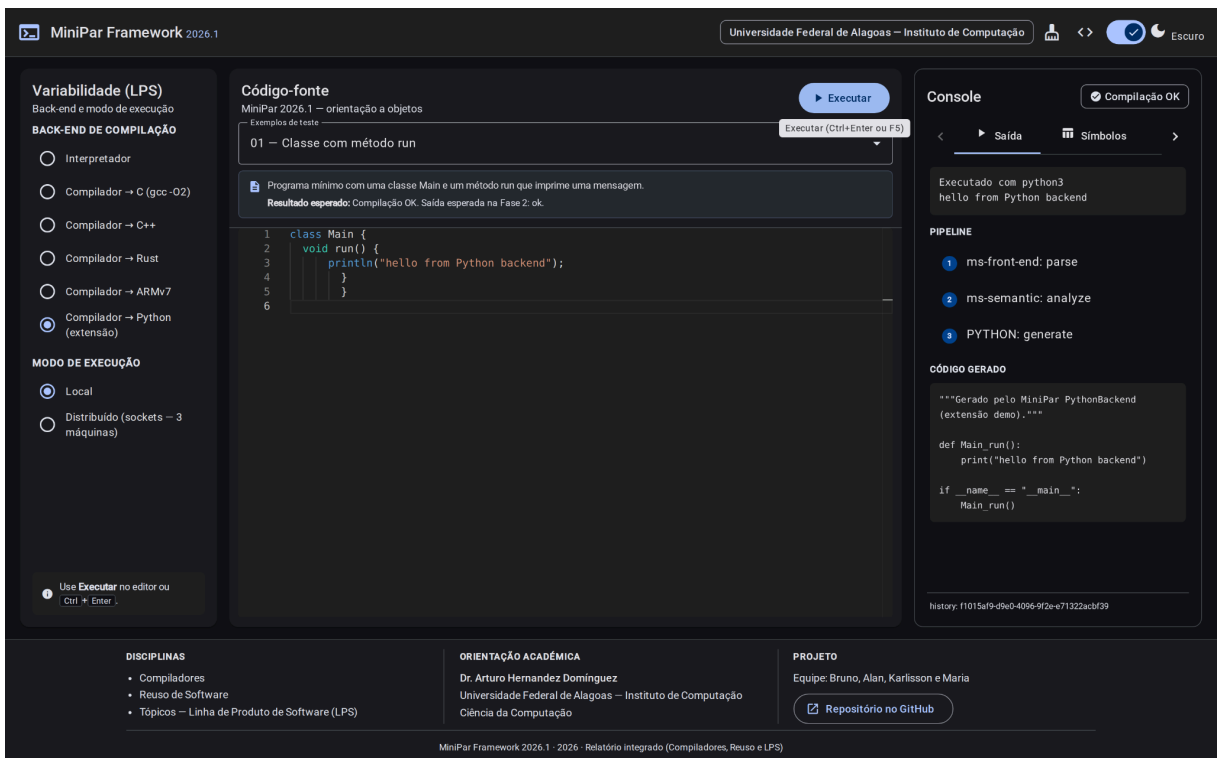
```
curl -X POST http://localhost:3000/api/v1/process \
-H "Content-Type: application/json" \
-d '{"sourceCode":"class Main { void run() { println(\"ok\"); } }",
    "targetVariability":"INTERPRETER",
    "executionMode":"DISTRIBUTED_SOCKETS"}'
# pipelineSteps inclui ms-parallel-coord: coordinate
# output agrega QuickSort (PC1), Matriz (PC2), Fatorial (PC3)
```

Saída real do teste distribuído:

```
=== Menu Coordenador - Paralelismo Real (3 Maquinas) ===
[PC1 c41305948de1:9001 quicksort] QuickSort PC1
[64,34,25,12,22,11,90] para [11,12,22,25,34,64,90]
[PC2 f7bc0d4810b7:9002 matrix] Matriz 2x2 PC2
A=[[1,2],[3,4]] x B=[[5,6],[7,8]] = [[19,22],[43,50]]
[PC3 167c730aac65:9003 factorial] Fatorial PC3
3628800
```



Pipeline E2E — variante C: passos parse, analyze, generate e saída gcc -O2.



Extensão PythonBackend — fixture 16_codegen_python.minipar; código Python gerado e execução com python3.

Variabilidade (LPS)

Back-end e modo de execução

BACK-END DE COMPILAÇÃO

- ☐ Interpretador
- ☐ Compilador → C (gcc -O2)
- ☐ Compilador → C++
- ☐ Compilador → Rust
- ☐ Compilador → ARMv7
- ☒ Compilador → Python (extensão)

MODO DE EXECUÇÃO

- ☒ Local
- ☐ Distribuído (sockets — 3 máquinas)



Use **Executar** no editor ou
Ctrl + Enter.


```
curl -X POST http://localhost:3000/api/v1/process \
-H "Content-Type: application/json" \
-d '{"sourceCode':"class Main { void run() { println(\"ok\"); } }",
    "targetVariability": "INTERPRETER",
    "executionMode": "DISTRIBUTED_SOCKETS"}'
# pipelineSteps inclui ms-parallel-coord: coordinate
# output agrega QuickSort (PC1), Matriz (PC2), Fatorial (PC3)
```

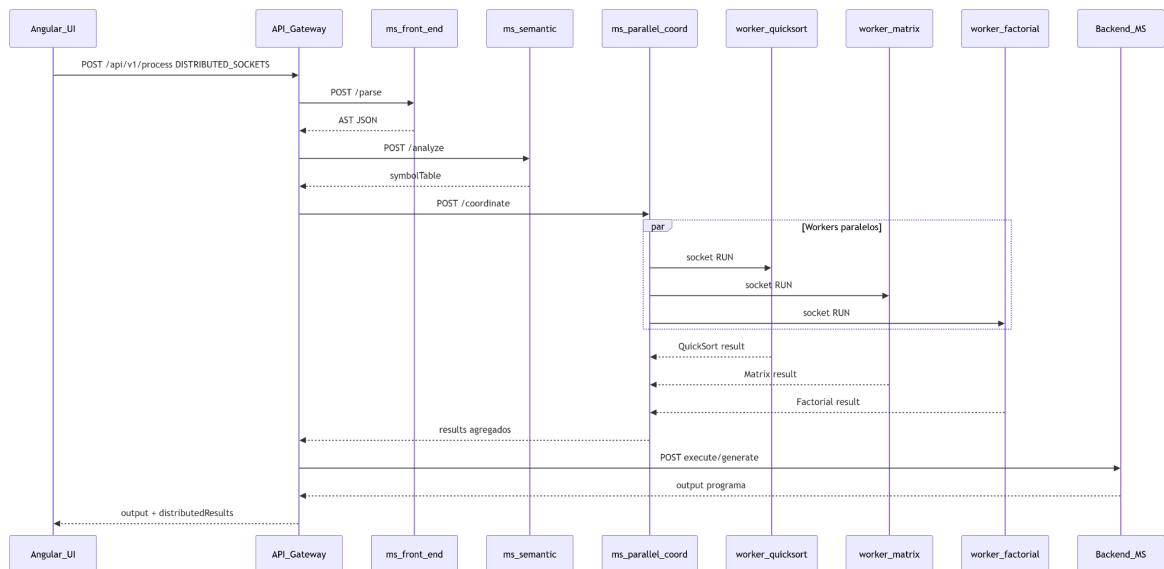


Figura 12: Sequência do teste de 3 máquinas (coordenador + workers socket).

Fixture: sources/examples/14_distributed_menu.minipar.

8.7 Simulação de Fractal

Implementou-se o tapete de Sierpinski recursivo em MiniPar OO (sources/examples/13_sierpinski.minipar): a classe Sierpinski utiliza o método recursivo isBlack e exibe uma matriz de caracteres * (sólido) e . (vazio) no Console da UI via interpretador. Ordem 3 produz matriz 27×27.

```
class Sierpinski {
    number isBlack(number x, number y, number n) {
        if (n == 1) { return 1; }
        number m = n / 3;
        if (x >= m && x < 2*m && y >= m && y < 2*m) { return 0; }
        return this.isBlack(x % m, y % m, m);
    }
    void draw(number order) { /* imprime matriz linha a linha */ }
}
class Main {
    void run() { Sierpinski s = new Sierpinski(); s.draw(27); }
}
```

8.8 Validação Automatizada

A validação de ponta a ponta executou-se mediante o script automatizado `validate-all.sh`, com a stack completa em execução via Docker Compose:

```
docker compose up --build -d
./scripts/validate-all.sh
# Resultado: dezesseis verificações aprovadas (validation-results.json)
```

A validação automatizada contemplou dezesseis verificações, incluindo os casos E1 a E12, a saúde do gateway, os endpoints de variantes e de saúde agregada dos microserviços.

ID	Caso	Status
GW	Gateway health	PASS
E1	08_interpreter_ok	PASS
E7	05_parse_extends_missing (erro parser)	PASS
E2	09_oo_new (interpretador)	PASS
E6	12_codegen_rust_stub (rustc)	PASS
E3	13_sierpinski (fractal)	PASS
E4/E10	Distribuído (legado + menu 14)	PASS
E8	04_semantic_extends_unknown	PASS
E5/E12	Codegen C (11, 09 OO)	PASS
E9	16_codegen_python	PASS
E11	15_channels (broker socket)	PASS
API	GET /variants, /recommendations	PASS

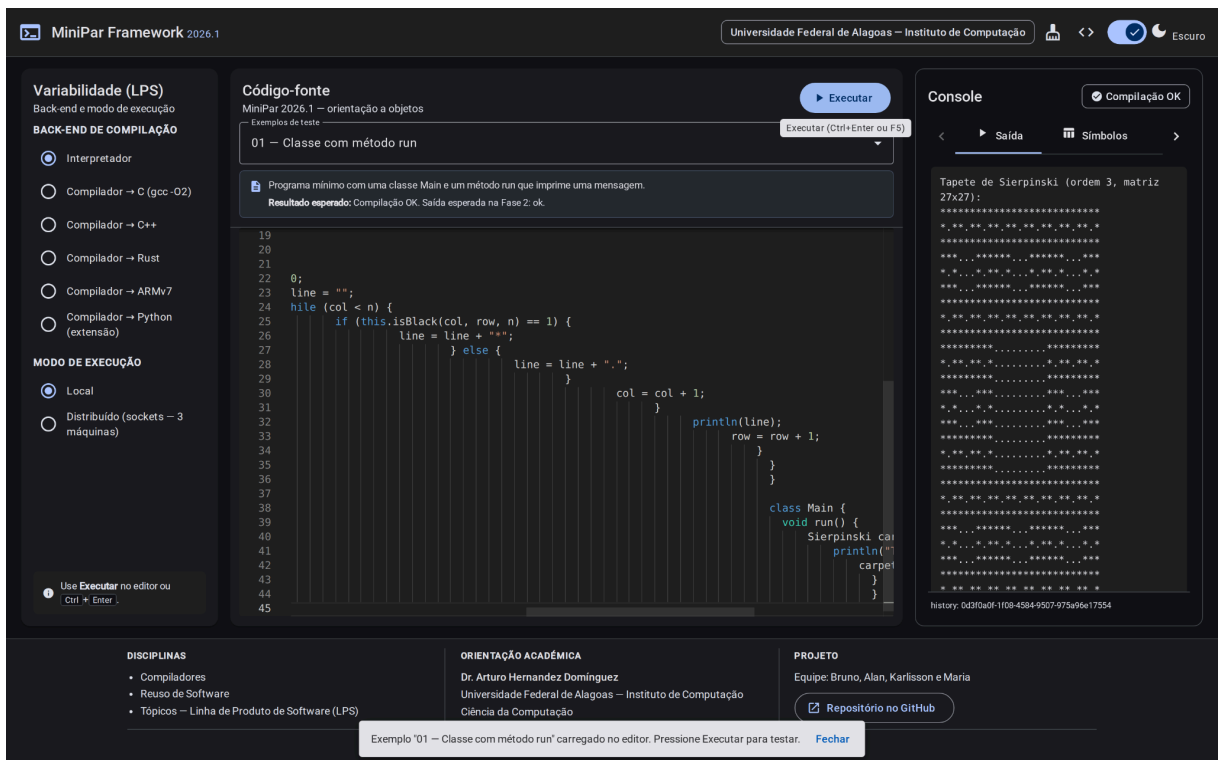
8.9 Auditoria de Conformidade

A conformidade com os requisitos das três disciplinas foi verificada por meio do protocolo descrito no Apêndice D. Em termos gerais, o pipeline orientado a objetos, o paralelismo por sockets TCP, o fractal de Sierpinski, os geradores com gcc e rustc, e a extensão em linguagem Python encontram-se implementados e validados.

8.10 Casos de Validação — Fase 3

Tabela 12: Casos de validação — Fase 3.

Arquivo	Modo	Resultado
13_sierpinski	INTERPRETER + LOCAL	Matriz fractal no Console
14_distributed_menu	INTERPRETER + DISTRIBUTED_SOCKETS	Resultados PC1–PC3
09_oo_new	C + LOCAL	gcc -O2 + saída woof (OO codegen)



Tapete de Sierpinski (ordem 3) — matriz 27 × 27 de * e . no Console da UI via interpretador.

9. Conclusão

O MiniPar Framework 2026.1 representa a convergência prática de três disciplinas em um único sistema coeso. Da perspectiva de compiladores, o projeto evolui a linguagem para OO, implementa paralelismo real via sockets e mantém uma pipeline completa de análise e geração de código. Da perspectiva de reuso de software e metodologias ágeis, o padrão Template Method estrutura a extensibilidade dos back-ends e o código base da versão anterior é incorporado sistematicamente. Da perspectiva de Tóp em eng de Software: projetando linhas de prod de software, o desacoplamento por microserviços materializa os pontos de

variação identificados, permitindo que diferentes produtos sejam derivados do mesmo framework sem modificações estruturais.

As fases 1–3 entregam pipeline E2E real, seis back-ends (interpretador, C/C++ com gcc -O2, Rust/ARM MVP, extensão Python com python3), teste de paralelismo distribuído em três workers socket e simulação do tapete de Sierpinski na interface web. A arquitetura modular adotada, aliada ao uso de Docker para isolamento e ao contrato AST JSON como interface entre serviços, assegura que cada componente possa evoluir independentemente.

9.1 Resposta à Crítica "Gerador versus Framework"

O artefato entregue constitui um framework de linha de produto de software, e não um gerador que emite repositórios independentes, descartando o núcleo em tempo de execução. Os trechos invariantes compreendem a orquestração do gateway, a sequência fixa de translate, os microsserviços de análise compartilhados e o contrato JSON da AST. As seis variantes de tradução implementadas funcionam como instâncias de referência, cujos pontos de adaptação já estão preenchidos para demonstrar o contrato. A extensão PythonBackend, validada com `16_codegen_python.minipar`, confirma que novas variantes podem ser acopladas sem alteração dos trechos invariantes.

9.2 Limitações e Trabalhos Futuros

- PAR local via threads; sockets apenas no teste dedicado de 3 máquinas.
- Workers executam Python fixo, não MiniPar compilado.
- Send/Receive parseados, mas não implementados no interpretador.
- Formalizar hotspots de lexer/parser alternativos (autômatos, bottom-up).
- UI dinâmica consumindo GET /api/v1/variants (endpoint já implementado).

9.3 Link para o Repositório (GitHub)

<https://github.com/brunog2/minipar-framework>

9.4 Link para o Ui de demonstração e Apresentação

<https://minipar-framework.vercel.app> (UI de demonstracao)

📄 Apresentação Minipar Framework 2026.1

10. Referências

- [1] Maciel, E. *MiniPar 2025.1 — interpretador orientado a objetos. Implementação de referência cl-minipar, turma Compiladores, UFAL, 2025.*
- [2] Rego, H. *Projeto compiladores MiniPar — geradores de código C e ARM, representação intermediária TAC e integração com gcc. Repositório projeto_compiladores, UFAL, 2025.*
- [3] Hernandez Dominguez, A. *Especificação do projeto integrado MiniPar Framework 2026.1. Documento da disciplina, UFAL, 2026.1.*
- [4] Pohl, K.; Böckle, G.; van der Linden, F. *Software Product Line Engineering: Foundations, Principles, and Techniques.* Springer, 2010.
- [5] Sommerville, I. *Software Engineering. 10th ed.* Pearson, 2016.
- [6] Gamma, E. et al. *Design Patterns: Elements of Reusable Object-Oriented Software.* Addison-Wesley, 1995.
- [7] Aho, A. V.; Lam, M. S.; Sethi, R.; Ullman, J. D. *Compiladores: Princípios, Técnicas e Ferramentas. 2. ed.* Pearson, 2008.
- [8] Fowler, M.; Lewis, J. *Microservices: a definition of this new architectural term.* martinowler.com, 2014. Disponível em: <https://martinfowler.com/articles/microservices.html>

Apêndice A — Registro Formal dos Sprints

Sprint	Período	Entregas principais	Responsáveis
1	2 a 4 de junho de 2026	Núcleo minipar-core; microserviços ms-front-end e ms-semantic; AST JSON; erros sintático e semântico	Equipe Compiladores
2	3 a 5 de junho de 2026	Módulo translation/; interpretador; geradores C, Rust e ARM; orquestração HTTP	Equipe Reuso
3	5 a 7 de junho de 2026	Coordenador distribuído; workers socket; fractal Sierpinski; extensão Python	Equipe LPS

Apêndice B — Contratos AST e REST

B.1 Contrato de Representação da AST em JSON

A comunicação entre os microserviços restringe-se a mensagens JSON. O documento raiz deve declarar obrigatoriamente o campo `type` com valor `Program`, seguido do vetor `declarations`. A serialização produzida pelo analisador sintático utiliza o módulo `ast_json.to_dict`; o analisador semântico e os geradores de código consomem a estrutura por meio de `ast_from_json.from_dict`.

Tipo	Campos e semântica
Program	Raiz do documento; contém <code>declarations[]</code>
ClassDecl	<code>name</code> , <code>extends</code> (opcional), <code>members[]</code>
MethodDecl	<code>name</code> , <code>returnType</code> , <code>parameters</code> , <code>body</code>
FuncDecl	Função global com tipo de retorno e corpo
ParBlock / SeqBlock	Blocos de paralelismo e sequência; <code>statements[]</code>
SendStmt / ReceiveStmt	Comunicação por canal; identificador do canal e expressão
ChannelDecl	Declaração <code>s_channel</code> ou <code>c_channel</code>
NewInstance	Instanciação <code>new</code> ; <code>className</code> , <code>arguments[]</code>
MethodCall	Chamada de método; receptor, nome e argumentos

```

{
  "type": "Program",
  "declarations": [{
    "type": "ClassDecl",
    "name": "Dog",
    "extends": "Animal",
    "members": [{
      "type": "MethodDecl",
      "name": "bark",
      "returnType": "void",
      "parameters": [],
      "body": { "type": "Block", "statements": [] }
    }]
  }]
}

```

B.2 Contrato de Resposta dos Geradores de Código

O resultado de qualquer variante de tradução materializa-se na estrutura `TranslationResult`, composta pelos campos `output` (texto emitido na execução), `code` (artefato gerado, quando aplicável) e `exit_code` (código de retorno do processo externo). O acoplamento entre microsserviços mantém-se reduzido: o analisador semântico não conhece o back-end de destino, e os geradores não reimplementam a análise léxica ou sintática.

Apêndice C — Processo de Instanciação de Nova Variante de Produto

A criação de uma nova configuração de produto sobre o MiniPar Framework pressupõe a reutilização integral do núcleo compartilhado (`minipar-core`, gateway central, microsserviços de análise léxica, sintática e semântica, e orquestração por contêineres). O desenvolvedor implementa exclusivamente os pontos de adaptação previstos pelo padrão Template Method, materializados nos métodos `emit` e `finalize` da classe derivada de `AbstractBackendTranslator`.

Em seguida, encapsula-se a lógica em um microsserviço autônomo que expõe o endpoint REST de geração ou execução, no mesmo molde do serviço `ms-codegen-c`. O registro da nova variante no vetor `BACKEND_REGISTRY`, a definição da variável de ambiente correspondente e a inclusão do serviço no arquivo de composição de contêineres completam a configuração em tempo de implantação.

Por fim, a interface de seleção de variabilidade deve contemplar o novo valor de `targetVariability`, de modo que o usuário final possa escolher a variante em tempo de execução sem alteração do algoritmo de orquestração. As instâncias de referência disponíveis compreendem o interpretador direto, o compilador C com `gcc -O2` e a extensão Python.

Apêndice D — Protocolo de Validação e Evidências de Execução

A validação de ponta a ponta do sistema executou-se mediante o script automatizado `validate-all.sh`, com a stack completa em execução via Docker Compose. Em quinze de junho de 2026, obteve-se aprovação em dezesseis verificações:

- Saída do interpretador para `08_interpreter_ok`
- Detecção de erro sintático em `05_parse_extends_missing`
- Compilação com `rustc` para `12_codegen_rust_stub`
- Geração do tapete de Sierpinski em `13_sierpinski`
- Coordenação distribuída nos modos legado e por menu MiniPar (`14_distributed_menu`)
- Verificação semântica de herança inválida
- Geração de código C e Python
- Teste de canais por socket TCP em `15_channels` (saída: 42)
- Consulta aos endpoints de variantes, recomendações e saúde agregada dos microsserviços

Saída textual do teste distribuído (Listagem 21 do relatório técnico):

```
=== Menu Coordenador - Paralelismo Real (3 Maquinas) ===
[PC1 c41305948de1:9001 quicksort] QuickSort PC1
[64,34,25,12,22,11,90] para [11,12,22,25,34,64,90]
[PC2 f7bc0d4810b7:9002 matrix] Matriz 2x2 PC2
A=[[1,2],[3,4]] x B=[[5,6],[7,8]] = [[19,22],[43,50]]
[PC3 167c730aac65:9003 factorial] Fatorial PC3
3628800
```

Saída textual do teste de canais (Listagem 22 do relatório técnico):

Apêndice E — Glossário

Trecho invariante (frozen-spot)

Segmento do framework que o desenvolvedor da instância não substitui, como o método `translate` e a orquestração do gateway.

Ponto de adaptação (hotspot)

Segmento variável preenchido na instância, em especial `emit` e `finalize`.

Instância de aplicação

Configuração da linha de produto sobre o chassi comum, compreendendo contêineres, pontos de adaptação implementados e registro de variantes.

Variabilidade

Dimensão configurável da linha de produto, como modo de execução, back-end de tradução e ambiente local ou distribuído.

Asset

Microserviço ou módulo reutilizável em múltiplas configurações de produto.

Binding time

Momento em que uma variante é vinculada: `implement time` (hotspots), `deploy time` (Docker Compose, `env vars`) ou `runtime` (campo `targetVariability` na API).

Princípio de Hollywood

Forma de inversão de controle na qual o framework invoca os pontos de adaptação do desenvolvedor, e não o inverso.